# claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_471a9cdd-d1d\agent-a8dfe81002dcd1b50.jsonl`
- SHA-256: `e23c7e02ad11d4795ed6333ef4429ee5b72a42856acb85d06d2045c5bce1b4d1`
- Source modified: `2026-06-10T09:38:51+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_471a9cdd-d1d`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

## Transcript

### 1. user (2026-06-10T09:33:15.142Z)

CONTEXT ? you are advising a real architecture decision for a solo-founder project (owner = avidullu).
Repos on disk (read-only; cite file paths for every claim):
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? FastAPI+SQLite+ffmpeg rally/highlight pipeline. Pluggable registries (segmenters/providers/validators/sports/annotations). Key seams: backend/pipeline/segmenters/base.py (registry), backend/pipeline/detectors/base.py (DetectorRunner ABC ? WASB/TrackNet run as WSL subprocesses from an EXTERNAL repo ~/models/WASB-SBDT), backend/providers/ (Gemini = external API). Eval stack backend/eval/ (metrics.match_intervals = the scoring spine, rally_seq_proto.py = the learned Gen-0 prototype seed, eval_partial_labels, calibrate_local LOVO, regression gate w/ tracked baseline in eval_baselines/). CI = .github/workflows/ci.yml (an import-smoke job WITHOUT the GPU stack + a full-suite job with CPU torch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? separate repo, the data system-of-record (golden labels, licensing gate, CVAT ingestion, 'curate export' manifest consumed by the indexer's batch eval).
- Also exists: private sports-obsreport repo = a shared report library consumed as a SOFT dependency by both repos (a working precedent for 'separate repo + consumed as dep').

THE DECISION: the owner is about to greenlight M0.4 (productionize rally_seq_proto into a Gen-0 TAL train?eval?ship-gate harness over ActionFormer/ASFormer (MIT), later Gen-2 = Qwen3-VL fine-tuned via ms-swift) and M0.1 (corpus spine + event-index contract, authored in the collector). QUESTION: should the model/training work live IN the indexer repo, or in a SEPARATE repo whose output is consumed by the indexer as just another segmenter/runner (like WASB and Gemini)? The owner PREFERS isolation ("iterate independently", cites the collector split as precedent) but asked for honest thoughts.

KNOWN FACTS to weigh (verified in a deep audit this week ? do not re-litigate, build on them):
- Cross-repo contract drift IS a real observed cost here: the collector's export header (start,end) vs the indexer's calibrate_wasb loader (start_time,end_time) forked into two incompatible golden-CSV conventions; tuned constants (TUNED/BASELINE) were duplicated-by-copy across files and drifted.
- Import-graph hygiene is a live concern: importing backend.main pulls torch/cv2 via the segmenter registration manifest (lazy-imports were deferred to the async-job slice).
- Authoritative plan docs: docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md ('envision now, build later' ? design every deferred seam up front; train EXPORT-CLEAN ops; on-device later = exported ONNX/CoreML/GGUF artifact + thin shell; the moat = consented dataset + eval harness, NOT the weights), docs/NEXT_STEPS.md, docs/PLATFORM_ARCHITECTURE.md.
- Guardrails: MIT/Apache-2.0/BSD only for code+data+weights deps; trained weights themselves = private/proprietary asset; minors excluded from training; per-user training opt-in; consent ledger planned in M0.1 schema.
- Quality state: learned LOVO mean 0.626 (n=6) vs heuristic 0.480; ship-gate F1@tIoU target still undefined.

- A repo RENAME off 'badminton' is already an owner action item ? repo topology is in flux anyway.
- Training runs on the owner's Windows box via WSL2 + RTX 2070 Super (8GB); Gen-2 may use rented cloud GPUs (Modal/RunPod per the study).
- Solo developer + AI agents do the work; there are ALREADY 4-5 repos (indexer, collector, rally-annotator, sports-obsreport).
RULES: read-only. Your final structured output is consumed programmatically.
ROLE: cross-examiner. Three position papers below. Attack the weakest claims of EACH against the actual repo (verify cited files where load-bearing). Surface what ALL of them missed (consider at least: the consent/minors guardrail and where the consent ledger lives; the GPU/WSL execution reality; agentic-development ergonomics ? AI agents working in parallel across repos vs one; the obsreport soft-dep precedent's actual maintenance experience; weights distribution if a hosted API ships; the cost of moving rally_seq_proto out of backend/eval given its test suite). Then give a decisive verdict: which position or precise hybrid is right for THIS project NOW, and what is the single contract that must be frozen first.
PAPER A (separate repo): {"position_summary":"The model/training work should live in a NEW separate repo (working name: sports-temporal-models) whose only product is a versioned, export-clean model artifact that the indexer consumes through a thin runner ? exactly the way it already consumes WASB (external repo ~/models/WASB-SBDT via subprocess, per backend/pipeline/detectors/wasb_runner.py) and Gemini (external API via the provider registry). The indexer's own architecture has already decided this question: the DetectorRunner ABC's docstring (backend/pipeline/detectors/base.py) is a written manual for consuming external inference substrates behind a healthcheck + artifact contract, and docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md mandates an export boundary (\"train with export-clean ops\", ONNX/CoreML/GGUF artifact + thin shell) regardless of repo topology. Putting ActionFormer/ASFormer and later ms-swift inside the indexer would be the first in-repo training dependency in a codebase whose CI deliberately proves it imports WITHOUT the GPU stack (.github/workflows/ci.yml import-smoke job) and whose import-graph hygiene is already a live concern. The known drift costs (the start,end vs start_time,end_time header fork) are real but they are arguments for a versioned contract with round-trip tests ? which M0.1 already mandates ? not for co-location, because the corpus lives in the collector either way, so the training harness sits between two repo boundaries no matter where its code lives.","key_arguments":[{"claim":"The indexer is architecturally a CONSUMER of external model substrates, and the seam for this is already built, documented, and proven twice. A separate training repo plugs into an existing socket; an in-repo trainer creates a new pattern.","evidence":"backend/pipeline/detectors/base.py lines 29-54 is literally titled 'How to add a new runner': subclass DetectorRunner, lazy-import heavy deps 'so importing the segmenter registry never pulls torch/tf', healthcheck = 'weights file exists and torch importable', and the named end-state is an ONNXRunner that 'loads weights locally (torch or onnx)'. backend/pipeline/detectors/wasb_runner.py consumes weights from an EXTERNAL repo (repo_dir='~/models/WASB-SBDT', weights_path config) ? the live shuttle substrate has never lived in this repo. backend/pipeline/segmenters/wasb_hybrid.py shows the consuming pattern end-to-end (healthcheck -> run_predict -> CSV -> windows -> DB). Gemini is the second precedent (backend/providers/, external API)."},{"claim":"The plan doc itself requires an artifact boundary; a repo boundary makes export-cleanliness structurally enforced instead of a convention that erodes. The on-device and rented-GPU futures both want a training repo that clones standalone.","evidence":"docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md: 'train with export-clean ops', on-device = 'exported ONNX/Core ML/ExecuTorch/GGUF artifact + a thin device shell... with zero Python', Gen-2 = 'Cloud train on Modal (no-access) / RunPod-Secure'. Cloning the FastAPI+SQLite+ffmpeg product repo (with its Gemini key handling, backend/main.py app, DB migrations) onto a rented RunPod box just to run ms-swift is pure liability; a training repo containing only datasets/train/eval/export is what you actually ship to a cloud GPU. The same artifact+manifest that serves the indexer IS the on-device deliverable later."},{"claim":"Dependency weight and CI guarantees diverge irreconcilably. The indexer CI's first job exists specifically to prove the codebase imports WITHOUT torch/cv2/numpy; M0.4/Gen-2 deps (ActionFormer's CUDA NMS extensions, ASFormer, ms-swift's transformers/deepspeed stack) are unbuildable on that CI and would either break the smoke guarantee or grow an unbounded stub list. Import-graph hygiene is ALREADY a live problem in-repo.","evidence":".github/workflows/ci.yml lines 16-31: import-smoke job 'Deliberately does NOT install cv2/torch/numpy'; full-suite job needs CPU-only torch + apt libgl just for opencv today. requirements.txt is 11 lines; ms-swift alone would multiply it. Known fact (verified audit): importing backend.main already pulls torch/cv2 via the segmenter registration manifest and lazy-imports were deferred. A separate repo gives each side the CI it needs: training repo = numpy/scipy unit tests + synthetic 1-epoch CPU train smoke + manifest schema validation; indexer CI = unchanged + one contract test loading a tiny committed fixture artifact through the

runner."},{"claim":"Cadence mismatch: training is experiment-shaped (config sweeps, broken intermediate states, baseline re-runs on the WSL/RTX box), the indexer is a product repo with a regression gate and a one-PR-per-item discipline. Co-location pollutes the product history or throttles experiments ? the owner's 'iterate independently' instinct matches the real workflow.","evidence":"CLAUDE.md workflow conventions ('Branch off master; ONE PR per work item; NO PR stacking'); backend/eval/regression.py + eval_baselines/regression_baseline.json (DEFAULT_BASELINE_PATH, regression.py line 33) is a product-quality gate, not an experiment log. rally_seq_proto.py's own header calls itself 'a directional PROOF-OF-SIGNAL, not a production model' ? the productionized version of that loop is dozens of runs/week, none of which should touch a repo whose master gates shipping."},{"claim":"The drift objection actually argues for contracts, not co-location: the corpus is authored in the COLLECTOR (M0.1), so train-time data crosses a repo boundary EVEN IF training lives in the indexer. The observed header fork happened on exactly that collector->indexer boundary while both endpoints were 'co-located enough' to share conventions informally ? informality, not separation, was the cause. The new boundary a third repo adds (artifact manifest) is the easiest kind to pin.","evidence":"docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md M0.1: 'Authored in sports-data-collector (system-of-record)... Acceptance: round-trip test'. backend/eval/batch.py already consumes the collector's 'curate export' manifest.json across the repo boundary. Known fact: the (start,end) vs (start_time,end_time) fork was fixed with normalization + an import guard (docs/NEXT_STEPS.md: 'now safe: import guard + --normalize, PR #15') ? i.e., the remedy was a versioned contract + tests, and it worked. A model artifact with schema_version + sha256 + feature-spec in its manifest is strictly more pinnable than a CSV header convention."},{"claim":"There is a working in-house precedent for 'separate repo consumed as a pinned soft dependency', including the editable-install pattern for fast local iteration ? so the solo-dev mechanics are already solved, not hypothetical.","evidence":"requirements.txt lines 12-19: obsreport consumed as 'obsreport @ git+ssh://...@v0.1.3' with documented local-dev fallback 'pip install -e ../sports-obsreport'; .github/workflows/ci.yml lines 50-52 documents the soft-dep-absent CI behavior (importorskip). The training repo's runtime subpackage follows this pattern verbatim: pinned tag in CI/production, editable sibling checkout during a hot iteration week."},{"claim":"IP/access asymmetry: the moat artifacts (consented corpus, ship-gate, weights) have a stricter governance surface than the product code, and the repo topology should reflect that ? especially with vendors already touching the data flow.","evidence":"docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md: 'The moat = the consented dataset + the eval harness, not the weights'; guardrails: weights = private/proprietary, consent ledger in M0.1 schema, minors excluded, per-user opt-in. Memory/golden-set-vendoring: third-party vendors (Roora) already participate in labeling. Keeping training+gates in a small private repo lets the indexer be opened to contractors/vendors/internship work (memory: sibling khelacharya layer) without exposing training configs, consent-gated data references, or weights."},{"claim":"The pending rename makes NOW the cheapest moment to set topology: create the training repo sport-agnostic from day one, and the indexer rename stays a pure rename instead of a rename + carve-out; extracting a training tree later means history surgery on the product repo.","evidence":"docs/NEXT_STEPS.md line 59: 'rename repo off badminton (multisport, owner action)'; docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md multisport section: 'ACTION: rename the repo off badminton ASAP (owner emphatic)'. Repo topology is in flux this week by the owner's own plan."}],"boundary_design":"NEW REPO: `sports-temporal-models` (private; sport-agnostic name consistent with sports-data-collector / sports-obsreport and the pending rename). Layout: `datasets/` (M0.1-JSONL -> ActivityNet-JSON / frame-label transcoders; consumes the collector's versioned corpus export ? collector remains the data system-of-record), `tal/` (Gen-0: the feature/training/threshold logic lifted from backend/eval/rally_seq_proto.py, wrappers over pinned MIT ActionFormer/ASFormer; later `vlm/` with ms-swift configs for Qwen3-VL), `harness/` (the M0.4 one-command train->eval->ship-gate; the F1@tIoU gate target is a VERSIONED config file here), `runtime/` (a tiny pip-installable package, numpy/onnxruntime only ? feature extraction + inference glue shared verbatim between train and serve; this kills the duplicate-by-copy drift class seen with TUNED/BASELINE constants), `export/` (emits the artifact). DELIVERABLE: a release bundle `rally-tal-gen0-vX.Y.Z` = model.onnx (export-clean per plan) + manifest.json carrying {schema_version, gen, sha256(weights), corpus_export_hash (the collector manifest it trained on), feature_spec (FEAT_NAMES + window/stride/fps-normalization ? exactly what silently forked before), input contract (trajectory CSV schema Frame,Visibility,X,Y from DetectorRunner), threshold/smoothing params, eval block (per-video LOVO + mean F1@tIoU vs the pinned golden set, gate verdict), license + consent attestations (no-paid-LLM-train, minors-excluded)}. WEIGHTS STORAGE: private GitHub Releases on the training repo (Gen-0 is ~1M params ? MBs; avoids git-lfs bandwidth billing and clone pollution); Gen-2 adapters go to a private HF repo or object storage with the manifest holding the URI+sha256. INDEXER SIDE: one thin `learned_tal` segmenter (BasePlaySegmenter) that reuses the existing DetectorRunner for the trajectory CSV, then runs the runtime package over it; healthcheck = artifact present + sha256 match + schema_version

compatible (the exact (ok,msg) contract of backend/pipeline/detectors/base.py); onnxruntime lazy-imported per the ABC's rule #4 so the import-smoke CI job is untouched; config pins `model_ref` + expected sha256. EVAL OWNERSHIP SPLIT: training repo owns the SHIP-GATE (model-selection eval against the collector golden set); indexer keeps backend/eval/regression.py + eval_baselines (end-to-end pipeline regression with the integrated runner) and backend/eval/batch.py (collector-manifest acceptance runs = the pre-promotion bridge test). The scorer spine metrics.match_intervals is the ONE shared piece: extract it as a micro-package (obsreport pattern) consumed pinned by both repos ? never duplicated by copy. CI: training repo = lint + numpy/scipy unit tests + synthetic 1-epoch CPU train smoke + manifest schema validation + a runtime-vs-recorded-snapshot contract test; indexer = existing jobs + one fixture-artifact contract test (a KB-scale committed dummy model). WHAT MOVES: rally_seq_proto's learn/threshold logic, distill_local's role (post-M0.3 purge), the LOVO training parts of calibrate_local. WHAT STAYS: all segmenters/runners, metrics (until the micro-package), eval_partial_labels, regression gate, batch eval, calibrate_wasb loaders. RENAME INTERACTION: create sports-temporal-models first with the sport-agnostic name; the indexer rename then has no carve-out entangled in it.","risks":["Cross-repo contract drift is an OBSERVED cost in this exact project, not a hypothetical: the collector/indexer golden-CSV header fork (start,end vs start_time,end_time) and the TUNED/BASELINE duplicate-by-copy drift both happened. A third repo adds a third boundary (the artifact manifest + feature spec). The mitigation (versioned manifest, shared runtime package, fixture contract tests, corpus_export_hash threading) is real engineering work that must land WITH M0.4, not after ? if the owner skips it under time pressure, the feature spec will fork exactly like the CSV header did, and a train/serve feature mismatch is silent quality corruption, the worst failure mode this project has.","Solo-dev overhead is real: this becomes repo #5-6; during the hot Gen-0 iteration phase, every experiment that needs an indexer-side change becomes two PRs plus a version bump, and release ceremony (tag, upload, pin) lands precisely when iteration speed matters most. The editable-install escape hatch (the documented obsreport `pip install -e ../` pattern) blunts this but requires discipline about what counts as a 'real' release.","The scoring spine is a single point of silent failure: if metrics.match_intervals ends up duplicated by copy instead of extracted/pinned, drift in the SCORER invalidates the ship-gate itself ? a worse outcome than either repo topology. The micro-package extraction is therefore not optional in this design, and it is extra up-front work.","Possible premature freezing: at n=6 videos and a linear prototype, the artifact contract (feature spec, manifest fields, even 'trajectory CSV in -> intervals out') may need weekly revision at first; contracts are cheapest to change inside one repo. Pre-1.0 schema_version bumps must stay socially cheap or the boundary will be resented and bypassed.","Data handling in the new repo needs care: training-repo tests want trajectories and golden labels as fixtures, but consented corpus data is governed by the collector's ledger ? committing convenience copies into the training repo would quietly fork the system-of-record the M0.1 design exists to prevent."],"what_would_change_my_mind":"Evidence that the Gen-0 loop is tightly coupled to indexer internals in practice: if, during the first month of M0.4, more than roughly a third of training-repo changes require a paired indexer PR (e.g., the harness turns out to need the SQLite DB, the per-video calibration code in backend/eval/calibrate_local.py, or the report stack rather than just the collector export + trajectory CSVs), then the boundary is in the wrong place and an in-repo top-level `training/` package with its own optional requirements file and an import-linter rule (backend must never import training) would deliver ~80% of the isolation at far lower ceremony. Likewise if the contract-discipline mitigations visibly fail ? copies of the feature code or metrics appearing in both repos despite the runtime-package plan ? that demonstrates the solo-dev ceremony budget is exceeded and consolidation is honest. Finally, if the owner resolves decision #4 by making the collector the system-of-record for training as well as data (two repos total, training beside the corpus it consumes), that adjacent-consolidation variant preserves every argument above except repo count, and I would prefer it over a third repo."}
PAPER B (monorepo): {"position_summary":"Keep Gen-0/Gen-1 model and training work IN the indexer, as an isolated top-level `training/` package with a one-way import rule, optional-dependency extras, and its own CI job ? and design the ARTIFACT contract (weights + metrics + gt_hash) as the seam you split along later. M0.4 is, by the project's own authoritative docs, an extension of `backend/eval/` (\"reusing metrics.match_intervals\", \"extend the existing backend/eval/ regression harness\"), and the prototype it productionizes imports four load-bearing indexer modules today; a repo split now severs working code, not a contract. The WASB/Gemini precedent doesn't apply ? those are externally-owned, frozen-interface artifacts, while Gen-0 is the fastest-churning code in the project (metric corrections moved the headline number 0.583?0.626 in one sweep this week), and this codebase's own audit shows cross-repo contracts here drift exactly when interfaces churn. The owner's isolation goal is real and achievable in-repo with enforceable mechanisms (an executable import-contract test, a torch-free smoke CI job, requirements-training.txt); the split should happen at Gen-2/ms-swift, when the

training environment becomes genuinely alien (rented cloud GPUs) and the M0.1 corpus contract has been hardened by a generation of real use ? splitting along a proven seam instead of a guessed one.","key_arguments":[{"claim":"The authoritative plans already specify M0.4 as an in-repo extension of backend/eval, and a prior documented decision rejects fracturing this codebase","evidence":"C:/Users/avidu/Projects/badminton-highlight-indexer/docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md M0.4: 'Productionize rally_seq_proto.py ... reusing metrics.match_intervals'; same doc (Language section): 'Reject a Go/Rust hot-path split ... it would fracture the one codebase holding the QA-state + eval harness + provider registry' ? the same argument applies to a Python repo split. C:/Users/avidu/Projects/badminton-highlight-indexer/docs/PLATFORM_ARCHITECTURE.md lines 400-413: 'The harness = thin glue over components we already have ... reuse the annotation registry + candidate_segments ... extend the existing backend/eval/ regression harness ... Register the result as the owned_vlm provider' ? the train?eval?gate?register loop is designed THROUGH the indexer's seams."},{"claim":"Training and serving share CODE (feature extraction, scoring, label loading), not just a contract ? a split forces either duplicate-by-copy (training/serving skew) or a new shared-lib repo for a solo dev who already maintains 4-5 repos","evidence":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/rally_seq_proto.py lines 38-41 imports backend.pipeline.detectors.tracknet_runner.TrackNetRunner (trajectory parsing), backend.eval.classifier.LogisticRegression, backend.eval.calibrate_wasb.load_golden_gts, backend.eval.metrics.score. Its build_frame_arrays/features functions must run IDENTICALLY at train time and at inference time inside the future LearnedSegmenter ? one copy in-repo makes skew structurally impossible. Same interlock in backend/eval/calibrate_local.py lines 29-32 and backend/eval/eval_partial_labels.py lines 33-37 (each reuses tracknet_runner, rally_gate, calibrate_wasb, metrics)."},{"claim":"The ship-gate can only execute where predictions, DB, baselines and golden providers live ? the indexer ? so a separate training repo would have to install the indexer to get its own ship/no-ship verdict, inverting the dependency the owner wants","evidence":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/regression.py lines 24-33: imports backend.infrastructure.database.Database, backend.eval.harness.get_predictions (reads the indexer DB), backend.annotations.annotation_registry, and the tracked baseline path eval_baselines/regression_baseline.json. M0.4 is explicitly train?eval?SHIP-GATE; the gate is the indexer-resident half."},{"claim":"The audit's own drift evidence argues against splitting the highest-churn code: this codebase forked a CSV header across ONE repo boundary (collector) and drifted constants across files within ONE repo ? drift cost scales with interface churn, and Gen-0's interfaces churn weekly","evidence":"Verified audit facts (given): collector export header (start,end) vs C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py load_golden_gts (start_time,end_time, lines 34-49); TUNED/BASELINE duplicated-by-copy (calibrate_wasb.py line 31 BASELINE, eval_partial_labels.py line 44 TUNED, calibrate_local.py line 36 BASELINE_LOCAL ? three sibling constants even within one repo). Churn rate: docs/NEXT_STEPS.md lines 12-13 ? midrank-AUC + fps-normalization corrections moved learned LOVO 0.583?0.626 in one sweep; the ship-gate F1@tIoU target is still undefined, so the metric/feature surface is still moving."},{"claim":"WASB and Gemini are the wrong precedent class for 'separate repo consumed as a runner': both are externally-owned, frozen-interface dependencies; even WASB's thin one-CSV subprocess contract needed hardening PRs (video_id keying, timeouts) this very week","evidence":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py lines 1-40: WASB = third-party repo at ~/models/WASB-SBDT run via WSL conda subprocess, contract = Frame,Visibility,X,Y CSV; backend/providers/ Gemini = external API. The in-house precedent for extracting OWN cross-cutting code is sports-obsreport ? a soft dep consumed via pinned git+ssh (requirements.txt lines 16-18, backend/reporting/__init__.py soft-import) ? which is precisely the cheap SPLIT-LATER mechanism available once a package boundary is clean, not an argument for splitting before the boundary exists."},{"claim":"Gen-0 has no environmental forcing function for a separate repo: it trains in minutes at ~4.5 GB on the same Windows/WSL box that runs the indexer, and the repo's CI already has both a torch-free boundary-guard job and a CPU-torch full-suite job ? the exact machinery needed to host training extras safely","evidence":"docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md ('trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params'); C:/Users/avidu/Projects/badminton-highlight-indexer/.github/workflows/ci.yml: import-smoke job (lines 20-31) deliberately installs no cv2/torch/numpy, full-suite job installs CPU torch (lines 46-49). Contrast Gen-2: ms-swift + rented A100/L40S (Modal/RunPod) ? THAT is when the environment becomes alien and the split earns its cost."},{"claim":"Split-now front-loads per-iteration cost at exactly the moment iteration speed is the product; split-later is mechanical if the one-way import rule holds","evidence":"Quality state (given): learned 0.626 vs heuristic 0.480 at n=6 with the gate target undefined ? the near-term work is tight change-feature?rerun-LOVO?check-gate loops over files that this week were edited together in single PRs

(backend/eval/{rally_seq_proto,calibrate_local,eval_partial_labels,metrics}.py per docs/NEXT_STEPS.md Pointers). Cross-repo, each such loop becomes a two-repo lockstep change + version pin bump. Conversely a clean `training/` package with no backend?training imports lifts out wholesale later (the obsreport pattern). Also: the repo rename off 'badminton' is already pending (OWNED_MODEL_IMPLEMENTATION_PLAN.md decision #5) ? adding a second repo now doubles identity churn mid-rename."}],"boundary_design":"ONE repo, TWO enforced boundaries ? a code boundary now, an artifact boundary forever. (1) Code boundary: new top-level package `training/` as a SIBLING of `backend/` (not inside it). One-way import rule: training?backend allowed (backend.eval.metrics, backend.eval.classifier/calibration, calibrate_wasb.load_golden_gts, TrackNetRunner's static parse/windowing helpers); backend?training FORBIDDEN, ever. Enforced executably, not socially (this is solo-dev + agents): extend tests/test_import_smoke.py with (a) assert `import backend.main` loads neither `training` nor torch/cv2 in sys.modules, (b) a grep-style test that no file under backend/ imports training. Shared feature extraction (rally_seq_proto's build_frame_arrays/features) is promoted to a small backend module (e.g. backend/eval/features.py or backend/features/) imported by BOTH the trainer and the serving segmenter ? one source of truth kills train/serve skew. (2) Dependencies: requirements.txt untouched; requirements-training.txt adds CUDA torch + ActionFormer/ASFormer (later ms-swift). CI gets a third job `training-suite` installing those extras and running training tests on fixture trajectories; the existing import-smoke job (no GPU stack) proves the boundary on every PR. (3) Artifact contract = the real seam (per 'envision now, build later'): one command `python -m training.speedrun --manifest ...` (nanochat-style per PLATFORM_ARCHITECTURE §training-harness) emits artifacts/<gen>/<run_id>/{weights (.json/.pt, export-clean ops, later .onnx), train_config.json, metrics.json (per-video F1@tIoU + LOVO), gt_hash, corpus_manifest_ref}. Weights = gitignored private asset (the moat is dataset+eval, not weights); metrics+gt_hash+manifest-ref ARE committed to eval_baselines/ next to regression_baseline.json. (4) Serving consumes ONLY the artifact: a `LearnedSegmenter` registered in segmenter_registry (backend/pipeline/segmenters/base.py) that lazy-imports its runtime inside methods, exactly per the documented DetectorRunner rule (backend/pipeline/detectors/base.py, 'lazy-import heavy native deps inside the methods'). Backend never imports training ? it loads a file, like it shells out to WASB. (5) Ownership: ship-gate logic lives once, in the indexer (backend/eval/regression.py + eval_baselines/), invoked by the training harness as a library call; M0.1 corpus spine + event-index + consent ledger stay in the COLLECTOR (system-of-record, per plan), consumed by training via the curate-export manifest as backend/eval/batch.py already does ? this time with a versioned-header contract test to prevent a repeat of the start,end fork. (6) Split trigger, written into docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md as an explicit gate so 'later' can't silently become 'never': WHEN Gen-2/ms-swift arrives (cloud GPUs, own docker images, multi-hour jobs) AND the collector's M0.1 JSONL + ActivityNet-JSON transcoder is the harness's sole data input, lift `training/` out wholesale as its own repo consumed obsreport-style (pinned tag), keeping metrics + the ship-gate in the indexer.","risks":["Import-graph creep is real here, not theoretical: backend/pipeline/segmenters/__init__.py already eagerly imports all five segmenters (which is how backend.main pulls torch/cv2 today), and the lazy-import fix was already deferred once to the async-job slice. A training/ package adds more heavy-dep surface; the full-suite CI job installs torch anyway, so ONLY the import-smoke job + the new contract test stand between the boundary and erosion ? and a test can be weakened in a refactor that an agent 'helpfully' simplifies.","Monorepo temptation entropy: with the code one import away, the path of least resistance for a solo dev (and especially for AI agents doing the work) is backend code reaching into training/ utilities or training reaching past the sanctioned seams into backend.main/segmenters. The one-way rule must be policed every PR forever; in a separate repo, the boundary polices itself.","'Split at Gen-2' may become 'split never': by then the in-repo harness will have momentum, shared fixtures, and CI wiring, and the extraction cost will be paid under deadline pressure (or skipped, leaving cloud-GPU training code awkwardly inside an indexing app repo).","Repo identity and ergonomics degrade: training PRs, app PRs, and eval PRs share one history, one issue space, and one growing test suite; the repo is mid-rename off 'badminton' and this position declines the one moment topology churn is cheapest, betting the artifact seam designed on paper now will actually be the right split line later."],"if_a second consumer of the trained head or harness appears earlier than expected (the khelacharya shot-intelligence layer, or collector-side pseudo-labeling of new sports), in-repo training makes the indexer a mandatory install for everyone, and the extraction has to happen under pressure rather than by plan."],"what_would_change_my_mind":"The core of my case is shared load-bearing code plus a co-located ship-gate; remove either and the position flips. Concretely: (a) if M0.1 lands such that the Gen-0/Gen-1 harness consumes ONLY the collector's JSONL corpus + ActivityNet-JSON transcoder ? no indexer feature extraction, no calibrate_wasb loaders, metrics consumed as a versioned package ? then the data contract already IS the boundary and a separate repo costs little; (b) if the model path goes straight to a pure-pixels VLM (no trajectory-feature code shared between train and serve), train/serve skew stops being a code-sharing

problem; (c) if a second consumer of the trained artifacts or harness materializes now (khelacharya, collector pseudo-labeling), a neutral home wins; (d) if Gen-0 training unexpectedly needs an alien environment (cloud GPUs, CUDA/python pins inexpressible as a requirements-training.txt extra on the WSL box); or (e) if the executable import-contract proves unenforceable in practice against agent-driven entropy ? caught violations more than once or twice ? I would concede the social-boundary experiment failed and split immediately along the artifact contract described above."}
PAPER C (ml-infra): {"position_summary":"The train/serve boundary that must hold is an ARTIFACT contract, not a repo boundary: serving consumes only a versioned bundle (weights + pinned featurizer + manifest), and the featurizer is single-source-of-truth code shared byte-for-byte between training and inference ? the fps-normalization bug just fixed in backend/eval/rally_seq_proto.py is exactly the skew class this prevents, and Google's Rules of ML #32 (\"re-use code between training and serving pipelines\") is the canonical statement of it. Given that, M0.4 Gen-0 should live IN the indexer repo as an isolated `training/` package (own requirements file, own CI lane, never imported by the serving path), not in a separate repo: the TAL head is not a black box like WASB or Gemini ? its input is the indexer's own featurization of the WASB trajectory, so a repo split puts the highest-skew-risk surface across the exact boundary where this project has already observed drift twice (start,end vs start_time,end_time; duplicated TUNED constants). The owner's real goal (\"iterate independently\") is achieved by dependency/import/CI isolation, not repo isolation. Defer a repo split to Gen-2 (Qwen3-VL via ms-swift on rented GPUs), when the artifact contract is proven, the featurizer genuinely differs (raw clips, not trajectory features), and the dependency tree is truly alien ? by then the manifest/registry seam makes extraction cheap, which is the plan's own \"envision now, build later\" principle applied to topology.","key_arguments":[{"claim":"The 'just another runner like WASB/Gemini' analogy fails technically: WASB's contract sits at the video boundary (video in -> Frame,Visibility,X,Y CSV out; DetectorRunner.run_predict) and Gemini's at video->segments, but the Gen-0 TAL head consumes FEATURES the indexer itself computes from the WASB trajectory ? the seam is mid-pipeline, and the featurizer IS the contract surface.","evidence":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/base.py (run_predict returns a trajectory CSV path; 'No change to eval/calibration code paths ? they consume the CSV + the static windowing helpers directly'); backend/eval/rally_seq_proto.py lines 38-41 imports four indexer modules (TrackNetRunner.parse_trajectory_csv, eval.classifier, eval.calibrate_wasb.load_golden_gts, eval.metrics.score) ? a separate training repo must either duplicate these (the observed drift class) or pin the indexer as a package, which it is not (it is an app)."},{"claim":"Train/serve skew is the dominant risk and it is featurizer-shaped: the same physical shuttle speed read ~2x smaller at 59.94fps than 30fps until the per-second normalization fix, and window params (win_sec=0.75, stride_sec=0.1), occlusion-gap logic, and post-processing (smooth=7, merge_gap=1.0, min_dur=1.0, tuned threshold) are all part of the model's identity ? any copy of this code in a second repo is a skew bug waiting to happen.","evidence":"backend/eval/rally_seq_proto.py: build_frame_arrays docstring lines 52-58 (the fps bug post-mortem), features() lines 91-111, probs_to_segments lines 138-159, train-tuned threshold line 189; FEAT_NAMES line 43."},{"claim":"Cross-repo contract drift is not hypothetical here ? it is live today: the collector exports 'start,end' CSVs while the indexer's golden loader reads 'start_time/end_time', a fork that survived both repos' green test suites because no single PR could see both sides. A solo dev + AI agents amplifies this: agents fix what the failing test in THEIR repo shows them; in-repo, a featurizer change breaks the round-trip test in the same PR.","evidence":"C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md:28 and src/cli/curate.py:552 ('start,end CSVs') vs C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py:34-49 (load_golden_gts parses start_time/end_time only) ? verified in both working trees on 2026-06-10."},{"claim":"Everything M0.4 productionizes already lives indexer-side, including a working proto-manifest and the exact hard-fail guard pattern the artifact contract needs: the classifier already serializes weights + normalization mean/std + feature_names to JSON, and regression.py already refuses incommensurable comparisons (gt_hash / IoU / detector mismatch) ? the ship-gate is an extension of existing product code, not new cross-repo machinery.","evidence":"backend/eval/classifier.py lines 86-114 (to_dict/save/load with w, b, mean, std, feature_names); backend/eval/regression.py lines 37-40 (gt_hash), 132-141 (incommensurability guards), 33 (DEFAULT_BASELINE_PATH='eval_baselines/regression_baseline.json' ? note: no eval_baselines/ dir is committed yet per git ls-files, so populating the tracked baseline is itself an M0.4 deliverable); backend/eval/metrics.py (the scorer spine)."},{"claim":"The import-hygiene worry argues for package isolation, not repo isolation: backend.main already pulls torch/cv2 through the eager segmenter manifest regardless of where training lives, and the CI already has the right two-lane shape (a no-GPU-stack import-smoke lane + a CPU-torch full lane) to enforce 'serving never imports training' with a test instead of

a repo wall.","evidence":"backend/pipeline/segmenters/__init__.py (eagerly imports yolo_hybrid/tracknet_hybrid/wasb_hybrid at registry load); .github/workflows/ci.yml lines 20-31 (import-smoke deliberately installs no cv2/torch/numpy) and 33-53 (full suite with CPU torch) ? adding a requirements-train.txt + an assertion that `import backend.main` loads no training module is a one-PR change."},{"claim":"Gen-0 training fits the product repo's footprint; Gen-2 is the natural split point: the plan itself says the head trains in minutes at ~4.5GB on the local 2070 Super with MIT deps (ActionFormer/ASFormer), while Gen-2 brings ms-swift, cloud-GPU launchers, and a different input modality ? and the plan's own 'envision now, build later' principle says design the seam (artifact contract) now and move code across it only when a milestone forces it.","evidence":"docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md: 'Design principle: envision now, build later' (lines 8-13), M0.4 (lines 127-132, trains on the GPU/WSL box), Gen-2 cloud training + unquantified budget (lines 144-149), 'train with export-clean ops' + on-device = exported artifact + thin shell (lines 85-95), 'the moat = the consented dataset + the eval harness, not the weights' (line 21)."}],"boundary_design":"THE INVARIANT (holds under any topology): serving loads ONLY a versioned artifact bundle, never training code at HEAD; the featurizer is one module imported by both sides; the manifest makes every assumption explicit and the loader/gate hard-fails on mismatch (the regression.py incommensurability pattern).\n\n1) ARTIFACT BUNDLE ? `artifacts/rally_tal_gen0/<semver>/` = weights file(s) + manifest.json, weights content-addressed (sha256). Manifest fields:\n- artifact: name, semver, created_at, training-code git SHA, sha256 per file, export format + ONNX opset (export-clean per OWNED_MODEL_IMPLEMENTATION_PLAN ? the on-device seam).\n- feature_schema: id=\"rally_seq\", integer version, feat_names (FEAT_NAMES from rally_seq_proto.py:43), window params {win_sec, stride_sec}, fps_semantics=\"per-second\" (the fixed bug class made explicit), occlusion max_gap rule, input contract = trajectory CSV (Frame,Visibility,X,Y) + REQUIRED runtime metadata {fps, frame_width} (both load-bearing), featurizer_fingerprint = sha256 of the featurizer module source.\n- normalization: per-feature mean/std frozen at train time (classifier.to_dict already does this ? never recompute at serve).\n- calibration: decision threshold (train-tuned, rally_seq_proto.py:189), smoothing window, merge_gap, min_dur ? post-processing is part of the model, not the consumer.\n- training_lineage: collector corpus-manifest ref + hash (the `curate export` manifest.json is the system-of-record handle), training video_ids (MD5) + split-by-match assignment, label source per video (human|vendor|pseudo), consent_fingerprint = hash over the M0.1 consent-ledger rows for every training video + attested booleans {no_minors, no_paid_llm_labels, training_opt_in} (M0.3 guardrails made machine-checkable).\n- eval_fingerprint: golden gt_hash (regression.py:37 pattern) of the n=6 corpus, protocol id (LOVO), achieved F1@tIoU table, baselines beaten {heuristic 0.480, learned proto 0.626}.\n- license_attestation: per-dependency {actionformer: MIT, asformer: MIT, ms-swift: Apache-2.0, ...} + detector_provenance {wasb_c3: UNRESOLVED} ? the gate reads this and refuses commercial-ship while C3 is open, even at passing F1.\nStorage: weights gitignored locally + private GitHub Releases (Gen-0 is single-digit MB; move to an HF private hub if Gen-2 adapters get large); manifest.json committed to a tracked dir (the eval_baselines/ pattern ? which regression.py references but which is not yet committed; creating it is part of M0.4). No git-lfs needed at Gen-0 scale.\n\n2) FEATURE CODE (train==serve by construction): promote build_frame_arrays + features out of backend/eval/rally_seq_proto.py into backend/features/rally_seq.py ? pure numpy/scipy, no torch, carrying a FEATURE_SCHEMA_VERSION constant. Imported by BOTH the new TAL segmenter (registered in segmenter_registry, lazy-importing weights inside methods per the DetectorRunner doc's own rule, base.py step 4) AND the training harness. The serve-time loader hard-fails when manifest.feature_schema.version != code constant or featurizer_fingerprint != current module hash.\n\n3) SHIP-GATE OWNER: the PRODUCT (indexer) owns the judge. Gate = extend backend/eval/regression.py: (a) verify manifest guards ? gt_hash vs the pinned golden set, feature-schema version, detector id, license attestation (C3), consent fingerprint vs the collector ledger export; (b) score via metrics.match_intervals on held-out videos; (c) compare vs the tracked baseline + the explicit F1@tIoU target (must be defined before M0.4 declares victory ? floor = heuristic 0.480, but set the real bar). The training harness runs the gate as local pre-flight, but admission to serving is the product-side gate against product-pinned scorer + collector-pinned labels. Rationale: no self-grading (the trainer optimizing the metric must not own the metric), the machinery already lives there, and the plan itself says the moat is the dataset + eval harness ? the judge is product infrastructure. The collector stays system-of-record for LABELS and consent (M0.1); the indexer owns SCORING and the pass/fail decision; the manifest pins both by hash.\n\n4) TOPOLOGY: M0.4 lands in the indexer as a top-level `training/` package: own requirements-train.txt (torch/ActionFormer/ASFormer never enter requirements.txt), a third CI lane for training tests, and an import-smoke assertion that `import backend.main` pulls no training module (extends the existing no-GPU-stack lane, ci.yml:20-31). M0.1 stays in the collector as planned ? but fix the one-contract problem first: a single shared loader/schema for the golden CSV (the start,end vs start_time,end_time fork) with a cross-repo round-trip test, since the manifest's gt_hash and consent_fingerprint both

flow through that export. Revisit a separate model repo at Gen-2: by then the artifact contract is exercised, and a training repo that produces conforming bundles can be split off without touching serving.","risks":["Dependency leakage by discipline, not wall: with training in-repo, an agent can import training code from the serving path 'just this once' and recreate skew through the backdoor ? the import-smoke assertion is a test, not a package boundary, and it must be written and kept green from the first M0.4 PR.","Repo growth and noise: training-iteration commits, configs, and experiment scripts intermix with product history; the pending rename off 'badminton' touches a bigger repo; run_all_tests.py and CI get a third lane to maintain.","Deferred-split migration cost: if Gen-2 does warrant a separate repo, moving training/ out later is real work (history, CI, paths) ? bounded by the artifact seam, but not free, and 'we'll split later' often becomes 'never'.","Overriding the owner's stated preference: he asked for isolation and cited a precedent he likes; if the in-repo recommendation is accepted grudgingly and the isolation tests are skipped to move fast, this position delivers the worst of both worlds (coupling without the contract).","The manifest contract is heavier than a 2-person-day prototype needs: for a 6-video corpus and a ~1M-param head there is a real risk of spending M0.4 budget on registry plumbing instead of the undefined ship-gate target and corpus growth, which the plan says are the actual bottlenecks."],"what_would_change_my_mind":"Any of: (1) Gen-0 scope actually requires the Gen-2 stack now ? ms-swift/transformers/deepspeed or cloud-launch scaffolding in the first milestone would make the dependency-tree argument for a separate repo immediate rather than deferred; (2) the indexer becomes a properly versioned, pip-installable package (setup.py exists but it is deployed as an app) so an external training repo could PIN the featurizer and scorer instead of copying them ? pinning is the mechanism that makes polyrepo ML safe, and its absence is half my case; (3) a second consumer of the model artifacts appears (another product, the khelacharya layer, a partner) ? a neutral model repo + registry then earns its coordination tax; (4) empirical evidence from this workflow that AI agents routinely break the serving path via training-side PRs despite the import-smoke guard ? that would show the test-not-wall mitigation failing in practice, and a repo boundary is the stronger wall."}

**2. user (2026-06-10T09:33:28.150Z)**

```
1       """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3       This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4       of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5       the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7       The ABC lets us:
8       - Keep the current WSL adapters working unchanged for Windows users.
9       - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11      - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14      Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15      - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16        Run detector, return the Windows-side path to the produced trajectory CSV
17        (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18        The caller (hybrid) is responsible for output dir; impls may stage the video
19        into WSL fs for perf.
20
21      - healthcheck() -> tuple[bool, str]
22        Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23        Message is logged (and can be surfaced to reports or UX in future).
24        Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26      Existing behavior is 100% preserved. The only additions are the inheritance and
27      the explicit common type for future implementers.
28
29      How to add a new runner (for docs / future contributors):
30      1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31      2. Implement run_predict and healthcheck (return (bool, str) tuple).
32      3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33         TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34         deliberately static and model-agnostic). If the new detector needs different
35         windowing, it can implement its own or we generalize later.
```

```
     4. Lazy-import heavy native deps inside the methods (so importing the segmenter
        registry never pulls torch/tf on a machine that doesn't have them).
     5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
        DetectorRunner instances (they already do the right thing via duck-typing + the
        type hints we add here).
     6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
        returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
     7. No change to eval/calibration code paths (they consume the CSV + the static
        windowing helpers directly).

     Optional Linux-native path (the main de-risk goal):
     A LinuxNativeRunner (or ONNXRunner) would:
     - Skip all wsl / _stage / conda activate.
     - Load weights locally (torch or onnx) or call a native binary.
     - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
     - healthcheck can just check "weights file exists and torch importable + CUDA?".
     This removes the WSL requirement for Linux users / future SaaS workers while
     keeping the exact same candidate quality characteristics (or better, once native
     weights are available).

     Error surfacing into reports:
     The healthcheck message is already logged by the hybrids on failure. The reporting
     harvest sees the outcome as "0 candidates" + detector name. When we wire richer
     segmenter Results (or pass detector_health into emit_indexer_report context), the
     (ok, msg) can be recorded explicitly under segmentation.details or a new detector
     stage. The ABC makes that future change local to one place.

     See also:
     - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
     - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
     - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
     """

     from abc import ABC, abstractmethod
     from typing import Optional, Tuple


     class DetectorRunner(ABC):
         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""

         @abstractmethod
         def run_predict(self, video_win_path: str, output_win_dir: str,
                         video_id: Optional[str] = None) -> Optional[str]:
             """Run inference and return path to trajectory CSV (or None on failure).

             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
             videos that happen to share a filename cannot reuse each other's frames or CSV.
             It is optional for back-compat; callers in the live pipeline always pass it.
             """
             pass

         @abstractmethod
         def healthcheck(self) -> Tuple[bool, str]:
             """Quick check if the runner (env, weights, etc.) is usable.

             Returns (ok, message). Never raises. Cheap to call before a long run.
             """
             pass
```

**3. user (2026-06-10T09:33:28.618Z)**

```
     """PROTOTYPE: a learned per-FRAME rally segmenter (vs the unsupervised windowing
heuristic).
```

```
    Motivation (2026-06-08 finding): WASB tracks the shuttle well on Kushagra (97% of GT
    rallies contain >=2 net-crossings) yet NO velocity-windowing config segments it
    (per-video F1 ceiling 0.163), while it works on Adarsh (0.73). The rally SIGNAL is in
    the trajectory; the hand-tuned heuristic just can't extract it across footage. This
    prototype tests the hypothesis that a LEARNED model on per-frame trajectory features
    recovers the rally segments ? including on the footage the heuristic fails on.

    Approach (deliberately small, dependency-light: numpy + scipy + the repo's pure-python
    LogisticRegression ? no torch/sklearn):
      1. Per-frame features over a local window from the WASB trajectory: visibility rate,
         mean/max shuttle speed (frame-width-normalized), net-crossing RATE, x/y spread.
         All are scale/camera-robust RATES (unlike a raw velocity threshold).
      2. Per-frame label = inside a human GT rally or not.
      3. LEAVE-ONE-VIDEO-OUT: train the classifier on the other match(es), test on the
         held-out one (the honest cross-footage number; no within-video leakage).
      4. Smooth probabilities -> threshold (tuned on TRAIN) -> contiguous in-rally segments
         -> merge tiny gaps / drop short -> score vs GT at IoU>=0.5 (same metric as the
heuristic).

    This is a directional PROOF-OF-SIGNAL, not a production model (n=2 videos, linear model,
    hand features). It exists to answer one question: is the rally boundary learnable from
    the trajectory where the heuristic fails? Compare its held-out F1 to the heuristic LOVO.

    Run:
        python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
    """
    from __future__ import annotations

    import json
    import argparse
    from typing import Dict, List, Tuple

    import numpy as np
    from scipy.ndimage import uniform_filter1d, maximum_filter1d
    from scipy.stats import rankdata

    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.eval.classifier import LogisticRegression
    from backend.eval.calibrate_wasb import load_golden_gts
    from backend.eval.metrics import score

    FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
    Interval = Tuple[float, float]


    def _spread(a: np.ndarray) -> float:
        return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0


    def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
        """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.

        Speed is normalized to fraction-of-frame-width PER SECOND (``* fps``), matching the
        heuristic windowing brain ? previously it was per-frame, so the same physical
shuttle
        speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
transfer
        on a corpus that mixes frame rates. Speed/crossings are also not computed across a
long
        occlusion gap (mirrors the heuristic resetting on invisibility)."""
        pts = sorted(points, key=lambda p: p.frame)
        N = (pts[-1].frame + 1) if pts else 1
        vis = np.zeros(N); x = np.zeros(N); y = np.zeros(N)
        for p in pts:
            if 0 <= p.frame < N and p.visible and p.x >= 0:
```

```
                vis[p.frame] = 1.0; x[p.frame] = float(p.x); y[p.frame] = float(p.y)

        mask = vis > 0
        play_is_x = _spread(x[mask]) >= _spread(y[mask])
        coord = x if play_is_x else y
        net = float(np.median(coord[mask])) if mask.any() else 0.0
        deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)

        speed = np.zeros(N); spd_mask = np.zeros(N); cross = np.zeros(N)
        max_gap = max(2, round(0.25 * fps))  # frames; a longer gap = occlusion break, not
motion
        prev = None
        for p in pts:
            f = p.frame
            if not (0 <= f < N and vis[f]):
                continue
            if prev is not None:
                df = f - prev
                if 0 < df <= max_gap:
                    speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
                    spd_mask[f] = 1.0
                    c, pc = coord[f] - net, coord[prev] - net
                    if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
                        cross[f] = 1.0
            prev = f
        return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)


    def features(arr: Dict, fps: float, win_sec: float = 0.75, stride_sec: float = 0.1):
        """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
        N = arr["N"]; fw = arr["fw"]
        size = max(3, int(win_sec * fps) * 2 + 1)
        vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))

        n = uniform_filter1d(vis, size, mode="constant")                      # mean
visibility (== vis_rate)
        sm = uniform_filter1d(spd_mask, size, mode="constant")
        spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
        spd_max = maximum_filter1d(speed, size, mode="constant")
        cross_rate = uniform_filter1d(cross, size, mode="constant") * fps       # crossings
/ second
        xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
        x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
        x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
        ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
        y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
        y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw

        idx = np.arange(0, N, max(1, int(stride_sec * fps)))
        X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
        return np.nan_to_num(X), idx / fps


    def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
        y = np.zeros(len(times))
        for i, t in enumerate(times):
            for s, e in gts:
                if s <= t <= e:
                    y[i] = 1.0
                    break
```

```
        return y


    def roc_auc(y: np.ndarray, p: np.ndarray) -> float:
        """Rank-based AUC (Mann-Whitney) with MIDRANK tie handling.

        Uses scipy.stats.rankdata (average method) so tied probabilities get averaged ranks
?
        without this, an all-tied classifier scored a spurious order-dependent 0.25..0.75
        instead of 0.5, and dead-time frames produce identical all-zero feature rows
(guaranteed
        ties), so the headline AUC carried an unquantified bias."""
        pos, neg = p[y > 0.5], p[y <= 0.5]
        if pos.size == 0 or neg.size == 0:
            return float("nan")
        ranks = rankdata(p)  # midranks (ties averaged)
        return float((ranks[y > 0.5].sum() - pos.size * (pos.size + 1) / 2) / (pos.size *
neg.size))


    def probs_to_segments(probs: np.ndarray, times: np.ndarray, threshold: float,
                          merge_gap: float = 1.0, min_dur: float = 1.0, smooth: int = 7) ->
List[Interval]:
        p = uniform_filter1d(probs, max(1, smooth), mode="nearest")
        on = p >= threshold
        runs: List[Interval] = []
        i, n = 0, len(on)
        while i < n:
            if on[i]:
                j = i
                while j + 1 < n and on[j + 1]:
                    j += 1
                runs.append((float(times[i]), float(times[j])))
                i = j + 1
            else:
                i += 1
        merged: List[Interval] = []
        for s, e in runs:
            if merged and s - merged[-1][1] <= merge_gap:
                merged[-1] = (merged[-1][0], e)
            else:
                merged.append((s, e))
        return [(s, e) for s, e in merged if e - s >= min_dur]


    def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
        f1s = []
        for v in vids:
            segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
            f1s.append(score(segs, v["gts"], 0.5).f1)
        return sum(f1s) / max(len(f1s), 1)


    def run(specs: List[Dict]) -> dict:
        data = {}
        for s in specs:
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
            X, times = features(arr, float(s["fps"]))
            gts = load_golden_gts(s["labels"])
            data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
gts}
        names = list(data)
        print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
videos) ===")
```

```
        print(f"features: {FEAT_NAMES}\n")

        results = []
        for test in names:
            train = [data[n] for n in names if n != test]
            Xtr = np.vstack([v["X"] for v in train]); ytr = np.concatenate([v["y"] for v in
train])
            clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
            clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
            # tune the decision threshold on TRAIN segment-F1 (no test leakage)
            thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
float(t)))
            v = data[test]
            probs = clf.predict_proba(v["X"])
            auc = roc_auc(v["y"], probs)
            segs = probs_to_segments(probs, v["times"], float(thr))
            sc = score(segs, v["gts"], 0.5)
            # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
ceiling
            # from the threshold-transfer gap. Optimistic ? for diagnosis only.
            othr = max(np.arange(0.2, 0.81, 0.05),
                       key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
            osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
            results.append({"video": test, "thr": float(thr), "auc": auc,
                            "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
                            "n_pred": len(segs), "n_gt": len(v["gts"]),
                            "oracle_thr": float(othr), "oracle_f1": osc.f1})
            print(f"[held-out {test}]  per-frame AUC {auc:.3f}  |  LOVO-threshold F1
{sc.f1:.3f} "
                  f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train)  |  "
                  f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
        mean_f1 = sum(r["f1"] for r in results) / len(results)
        print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
        return {"per_video": results, "mean_f1": mean_f1}


    def main() -> None:
        ap = argparse.ArgumentParser(description=__doc__)
        ap.add_argument("--manifest", required=True, help="JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
        args = ap.parse_args()
        with open(args.manifest, encoding="utf-8") as f:
            run(json.load(f))


    if __name__ == "__main__":
        main()
```

**4. user (2026-06-10T09:33:29.556Z)**

```
      name: CI

      on:
        push:
          branches: [master]
        pull_request:

      permissions:
        contents: read

      concurrency:
        group: ci-${{ github.ref }}
```

```
13        cancel-in-progress: true
14
15      jobs:
16        # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
17        # module-level import error shipping while the suite stays green.
18        # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
19        # on machines without the GPU stack (test_import_smoke stubs what's absent).
20        import-smoke:
21          name: Syntax sweep + import smoke (no GPU stack)
22          runs-on: ubuntu-latest
23          steps:
24            - uses: actions/checkout@v4
25            - uses: actions/setup-python@v5
26              with:
27                python-version: "3.13"
28            - name: Install minimal deps
29              run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
30            - name: Run import smoke tests
31              run: python -m pytest tests/test_import_smoke.py -v
32
33        full-suite:
34          name: Full test suite (offline, mocked)
35          runs-on: ubuntu-latest
36          steps:
37            - uses: actions/checkout@v4
38            - uses: actions/setup-python@v5
39              with:
40                python-version: "3.13"
41                cache: pip
42            - name: System libs for opencv-python
43              run: |
44                sudo apt-get update
45                sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
46            - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
47              run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
48            - name: Install requirements
49              run: python -m pip install -r requirements.txt pytest httpx  # httpx: fastapi
TestClient
50            # obsreport (private, soft dep) is absent here: reporting tests skip via
51            # pytest.importorskip ? run_all_tests.py prints which.
52            - name: Run full suite
53              run: python run_all_tests.py
54
```

**5. user (2026-06-10T09:33:29.624Z)**

```
1       fastapi>=0.111.0
2       uvicorn>=0.30.1
3       opencv-python>=4.9.0.80
4       numpy>=1.26.4
5       pydantic>=2.7.1
6       jinja2>=3.1.4
7       python-dotenv>=1.0.0
8       google-genai>=2.4.0
9       torch>=2.12.0
10      torchvision>=0.27.0
11      supervision>=0.21.0,<0.30.0
12
13      # Per-video observability reports (SOFT dependency: the pipeline degrades to a
14      # no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once
15      # the private repo is published, e.g.:
16      #   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
```

```
17      # For local development install it editable from a sibling checkout:
18      #   pip install -e ../sports-obsreport --no-build-isolation
19
```

**6. user (2026-06-10T09:33:35.509Z)**

```
1       """Regression runner ? verify the production tool against golden-set ground truth.
2
3       Implements the explicit ask: *"given a video, get a golden set created and verify
4       its precision and recall against the production tool."* This is deliberately a thin
5       wrapper over machinery that already exists:
6
7           ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8           predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
9           comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
10          baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12      A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13      baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14      later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15      independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16      """
17      import os
18      import json
19      import hashlib
20      import logging
21      from dataclasses import dataclass, asdict
22      from typing import Dict, List, Optional
23
24      from backend.infrastructure.database import Database
25      from backend.eval import metrics
26      from backend.eval.harness import get_predictions
27      from backend.annotations import annotation_registry
28
29      logger = logging.getLogger(__name__)
30
31      # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32      # actually load a committed baseline to compare against.
33      DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34      DEFAULT_TOLERANCE = 0.05
35
36
37      def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38          """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39          norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40          return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43      @dataclass
44      class RegressionResult:
45          video_id: str
46          stage: str
47          iou_threshold: float
48          precision: float
49          recall: float
50          f1: float
51          # Baseline values compared against (None when no baseline existed yet).
52          baseline_precision: Optional[float]
53          baseline_recall: Optional[float]
54          tolerance: float
55          regressed: bool
56          reasons: List[str]
57          # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58          # from "nothing to compare" so the CLI can refuse to silently green-light the
```

```
latter.
59          has_baseline: bool = False
60          gt_hash: Optional[str] = None
61
62          def as_dict(self) -> dict:
63              return asdict(self)
64
65
66      # ---------------------------------------------------------------- baseline store
67
68      def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69          """Load the per-video baseline snapshot file (returns {} if absent)."""
70          if not os.path.isfile(path):
71              return {}
72          with open(path) as f:
73              return json.load(f)
74
75
76      def _baseline_key(video_id: str, stage: str) -> str:
77          return f"{video_id}::{stage}"
78
79
80      def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                        f1: float, path: str = DEFAULT_BASELINE_PATH,
82                        iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                        gt_fingerprint: Optional[str] = None) -> None:
84          """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86          Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87          later run computed under different settings (or against different labels) is
detected
88          as incommensurable rather than silently compared.
89          """
90          baselines = load_baselines(path)
91          baselines[_baseline_key(video_id, stage)] = {
92              "video_id": video_id, "stage": stage,
93              "precision": precision, "recall": recall, "f1": f1,
94              "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95          }
96          os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97          with open(path, "w") as f:
98              json.dump(baselines, f, indent=2, sort_keys=True)
99          logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)
100
101
102     # ---------------------------------------------------------------- the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105                 stage: str = "final", iou_threshold: float = 0.5,
106                 detector: Optional[str] = None,
107                 tolerance: float = DEFAULT_TOLERANCE,
108                 baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
baseline.
110
111         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113         precision OR recall falls more than `tolerance` below the snapshotted baseline.
114         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115         caller can choose to snapshot the current numbers via `save_baseline`.
116         """
117         preds = get_predictions(db, video_id, stage, detector=detector)
118         s = metrics.score(preds, ground_truth, iou_threshold)
119         fp = gt_hash(ground_truth)
```

```
120
121         baselines = load_baselines(baseline_path)
122         b = baselines.get(_baseline_key(video_id, stage))
123
124         reasons: List[str] = []
125         regressed = False
126         has_baseline = b is not None
127         base_p = base_r = None
128         if b is not None:
129             base_p, base_r = b["precision"], b["recall"]
130             # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131             # against different labels is not a valid reference ? flag it instead of
trusting it.
132             b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133             if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134                 regressed = True
135                 reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136             if b_det is not None and b_det != detector:
137                 regressed = True
138                 reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139             if b_hash is not None and b_hash != fp:
140                 regressed = True
141                 reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142             if s.precision < base_p - tolerance:
143                 regressed = True
144                 reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145             if s.recall < base_r - tolerance:
146                 regressed = True
147                 reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151         return RegressionResult(
152             video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153             precision=s.precision, recall=s.recall, f1=s.f1,
154             baseline_precision=base_p, baseline_recall=base_r,
155             tolerance=tolerance, regressed=regressed, reasons=reasons,
156             has_baseline=has_baseline, gt_hash=fp,
157         )
158
159
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                               tier: str = "file", stage: str = "final",
162                               iou_threshold: float = 0.5,
163                               detector: Optional[str] = None,
164                               tolerance: float = DEFAULT_TOLERANCE,
165                               baseline_path: str = DEFAULT_BASELINE_PATH,
166                               guideline: Optional[str] = None,
167                               sport: str = "badminton",
168                               provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
```

```
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                        detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "".join(f"\n   - {why}" for why in r.reasons)
189
```

**7. user (2026-06-10T09:33:36.501Z)**

```
1       """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3       Wires the already-tested pieces together:
4         WASB trajectory CSV (Frame,Visibility,X,Y)
5           -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
6           -> backend.eval.calibration (improvement_report / cross_validate)
7
8       Run (after a full-video WASB pass exists):
9           python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10              --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an **optimistic** upper bound. The trustworthy number is the
14      cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17      from __future__ import annotations
18
19      import csv
20      import argparse
21      from typing import List, Tuple, Callable
22
23      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
24      from backend.eval.calibration import (
25          select_best, improvement_report, cross_validate, evaluate,
26      )
27
28      Interval = Tuple[float, float]
29      Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap,
min_window_duration)
30
31      BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults
32
33
34      def load_golden_gts(csv_path: str) -> List[Interval]:
35          """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36          gts: List[Interval] = []
37          with open(csv_path, newline="", encoding="utf-8") as f:
38              reader = csv.DictReader(f)
39              reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
40              for row in reader:
41                  row = {(k or "").strip().lower(): v for k, v in row.items()}
42                  try:
43                      s = float((row.get("start_time") or "").strip())
```

```
                    e = float((row.get("end_time") or "").strip())
                except (TypeError, ValueError):
                    continue
                if e > s:
                    gts.append((s, e))
        return sorted(gts)


    def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
        """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        def predict_fn(cfg: Config) -> List[Interval]:
            vt, mg, mwd = cfg
            wins = TrackNetRunner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width,
                velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            )
            return [(w["start"], w["end"]) for w in wins]

        return predict_fn


    def default_grid() -> List[Config]:
        return [(vt, mg, mwd)
                for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
                for mg in (1.0, 2.0, 3.0)
                for mwd in (1.0, 2.0, 3.0)]


    def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
            iou: float = 0.5) -> dict:
        gts = load_golden_gts(labels_csv)
        if not gts:
            raise SystemExit(f"no usable golden rallies in {labels_csv}")
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        grid = default_grid()
```

**8. user (2026-06-10T09:33:37.116Z)**

```
    from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
    from backend.pipeline.segmenters.motion import MotionPlaySegmenter
    from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
    from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
    from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
    from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter

    __all__ = [
        "segmenter_registry",
        "BasePlaySegmenter",
        "MotionPlaySegmenter",
        "GeminiPlaySegmenter",
        "HybridYoloSegmenter",
        "TrackNetHybridSegmenter",
        "WasbHybridSegmenter",
    ]

```

**9. user (2026-06-10T09:33:45.142Z)**

```
docs/archives/checkpoints/2026-06-eval-baseline/README.md
---
no eval_baselines dir in working tree
```

**10. user (2026-06-10T09:33:45.499Z)**

```
1       # Owned-model implementation plan (roadmap) ? DRAFT for owner review
2
3       **Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner
greenlights a specific
4       milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a
training source).
5       **Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO +
learned prototype
6       (`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy
workflow (2026-06-09).
7
8       ## Design principle (owner directive): *envision now, build later*
9       Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be
**architecturally envisioned
10      now** so we never hit a mammoth refactor. We **design the seams** (data contracts,
export boundaries, the
11      event store) up front and **implement behind them** when each milestone is greenlit.
Punting a *feature* is fine;
12      punting its *seam* is not.
13
14      ## North Star (reframed per PR #61 review)
15      - **Immediate deliverable:** a **training framework + harness** that produces a model
with **very high precision/
16        recall for extracting highlights + rallies from a clip**. *Quality is the first-order
objective.*
17      - **Eventual product:** a **single, sport-agnostic, conversational** video model ?
upload a clip, then ask
18        contextual questions ("longest rally", "make a clip of all service faults", "rallies
over 20 shots").
19      - **On-device is LATER** (a model-compression goal so power users save upload
cost/hassle) ? definitely on the
20        roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
21      - **The moat = the consented dataset + the eval harness, not the weights.**
22
23      ## Hard guardrails (non-negotiable ? "never corrupted")
24      - **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT
/ Apache-2.0 / BSD only.**
25        (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-
restrictive: Gemma 3 (viral),
26        **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU
cap), Commons-Clause repos.
27      - **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public
grounding-CoT datasets that
28        were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision
(a real, binding cost).
29      - **Exclude minors; per-user training data needs a separate explicit opt-in; split data
by match.**
30      - ? **Base-model pretraining provenance is unauditable for *every* open VLM (incl.
Qwen3-VL).** Our "no paid-LLM
31        training" rule is enforceable at *our fine-tuning data* layer, not the base's
pretraining ? an **explicit owner
32        risk-acceptance** is required to use any open VLM. (Governed by the base's license,
not our pipeline.)
33
34      ## The model strategy (verified 2026-06-09)
35      **Framework ? base** (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge
(runs SFT/GRPO, emits
36      weights); a *base* is the metal you forge. You pick one framework and feed it any clean
base.
37      - **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio
+ timestamp-grounding +
38        GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core;
*avoid its AGPL Studio UI*)
```

as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-time snapshot ? re-verify at impl.
- **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-video context, purpose-built
**timestamp emission** (second-level localization), multi-turn video chat. Re-confirm the exact size's LICENSE
pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but its Prohibited-Use posture is
unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not the primary extractor.
**InternVL3 (MIT/Apache)** = clean fallback if a Qwen blocker appears. ? base swap is cheap *only within
timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-specific).
- **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss strict temporal boundaries**
(best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse regions, not accurate boundaries").
**Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-risked on our corpus (held-out
AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params (trivially compressible later),
and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So: **head now ? VLM later**,
strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes, head refines boundaries) over
wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur footage may differ ? re-measure.)*

## Conversational video-QA ? build the bookkeeping NOW, punt the model
**Punt the conversational *feature*** until extraction quality is solid (current LOVO ~0.583 ? "very high"; a chat UI
over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn** (expensive; bad at exact
counting; the strong teachers are license-forbidden). **But build the *seam* now** (the brief requires it):
- **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits structured timestamped
spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-SQL + ffmpeg clip-cut). The
cited queries ("longest rally", "all service faults", "rallies > 20 shots") are **structured-numeric ? SQL, not a
VLM job** (SQL counts exactly where VLMs hallucinate).
- **Designed-now seam:** make the **per-video event-index schema a first-class data contract** alongside the M0.1
corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count, ending_reason/fault_tags,
derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy (e.g. `service_fault`) up front**
even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration): `play_segments` today has
`video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights` isn't a table yet ? so it's an
*extension with new columns + a highlights table*, not a no-op. **Punt** the NL parser/agent until quality clears the bar.
- Paid Gemini may answer genuinely open-ended edge questions **at inference** over our structured index (allowed ?
it reasons over our data, it is not trained on its outputs).

## Multisport (single sport-agnostic model)
- **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned across sports (sport = an input
feature / per-sport calibration). The collector + indexer are already sport-generic. ? **ACTION: rename the repo off
"badminton" ASAP** (owner emphatic) ? consistent with this.

```
75      - **What multisport changes = DATA + DETECTORS (scale up, the accepted "good
problem"):** each new sport needs its own
76        labeled matches (split by match) + a **clean trajectory detector** to feed the head's
features. **The binding blocker
77        is per-sport detectors:** WASB (badminton) has the unresolved **C3** checkpoint-
provenance issue, and **clean MIT/
78        Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet
identified.** Sequence by public-data
79        richness: **badminton ? tennis/TT ? pickleball/padel.**
80      - ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not
proven ? our corpus is badminton-only,
81        and the evidence once cited for it (RacketVision) was *refuted on verification* (it's
ball-tracking, not temporal). The
82        concept is sound on first principles (rally signal = a racquet-sport invariant) but
must be validated once a 2nd-sport
83        corpus exists. Don't cite RacketVision as evidence.
84
85      ## Language ? Python everywhere; on-device runtime is a deferred, layer-local export
shell
86      - **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer);
offline/batch, no latency hot path.
87      - **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-
process (ffmpeg, WASB-in-WSL, cloud
88        GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ?
single-digit-ms wins dwarfed by
89        ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval
harness + provider registry.
90      - **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate
our dev language:** train in
91        Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device
shell (Swift/Kotlin/C++/Rust)
92        runs the artifact with zero Python. **The one concrete constraint now: train with
export-clean ops** so the compression
93        target stays reachable without re-architecting. (On-device base stays on the
Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
94      - Escalation order if a real Python CPU hot path ever appears: free-threading ?
numpy/vectorize ? Cython/PyO3 ? only then a
95        separate service. **None exists today.**
96
97      ---
98
99      ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101     ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) ·
*greenlight item*
102     Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103     ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104     shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105     labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL
ActivityNet-JSON · VLM clip?QA),
106     one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107     **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109     ### M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
110     6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111     "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
112     data + calibration, not method.** Keep growing per new sport.
113
```

### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) · *greenlight item*
- **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training violation is
  **`backend/eval/distill_local.py`** (it trains a classifier on Gemini-silver CSV labels ? `output/local_rally_model.json`)
  and the threshold distillation in `calibrate_local.py`. (NB: `backend/eval/training.py::label_candidates` is a *pure,
  source-agnostic* function ? there is no store by that name; the earlier "training.label_candidates" pointer was wrong.)
  Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the path that regenerates them is one
  command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini = eval/reference/fallback only.
- **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint provenance is a **commercial-ship
  blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a clean MIT/Apache detector swap.
  ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our temporal labels can't train the
  shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies per sport** (incl. any TT-on-WASB use).
  Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).

### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over **ActionFormer (MIT, 1:1
rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not* the Commons-Clause repo),
reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item): a FLOOR of "beat the honest n=6
heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU** target for "very high P/R" (e.g.
F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory. Trains on the GPU/WSL box.

---

## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it becomes the rally/highlight
extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no overfitting to declare victory.

## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus + explicit go)
- **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a promising cue here** ? our E1 probe
  (PR #35) found it weak (~2.2× discrimination, AUC ~0.6) and the foreground-audio idea also failed this session; keep it
  parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet first).
- **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a temporal-IoU-vs-golden reward,
  our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on Modal (no-access) / RunPod-Secure.
- **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-hosted VLM vs Gemini) **+ a real
  VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples use 6?8 GPUs ? **budget is the
  largest open risk and is currently unquantified**). Don't stand up an owned serving stack until these clear.

---

```
152     ## Decisions / status
153     | # | item | status |
154     |---|---|---|
155     | 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
156     | 2 | WASB C3 ? first-class action item (own-weights / clean swap) | **owner: yes** ? |
157     | 3 | Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights) | **owner: yes** ?
|
158     | 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
159     | 5 | Rename repo off "badminton" | **owner action** ? |
160     | 6 | Conversational-QA: build seam now, punt model | **rec accepted?** |
161
162     ## Open risks / unknowns (none block Phase-0 research)
163     Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R"
target undefined · per-sport clean
164     trajectory detectors not yet identified · base-model pretraining provenance unauditable
(risk-acceptance) · real VLM
165     VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-
train (minors-excluded) for cloud
166     training · on-device 4B latency on 8 GB unmeasured.
167
168     ---
169     *Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
170     next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*
171
```

**11. user (2026-06-10T09:33:55.757Z)**

```
1       # Next steps (live) ? owned-model + multisport
2
3       **Updated:** 2026-06-10 (post audit-hardening sweep). **Authoritative roadmap:**
4       `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`. **Latest blow-by-blow:** memory `current-
state.md` / `session-handoff.md`.
5       This file = the prioritized "what to do next," refreshed each session.
6
7       > **2026-06-10 sweep (12 PRs, both repos, all merged):** CI now exists in both repos;
master un-broken (compiler
8       > SyntaxError fix); detector artifacts keyed by video_id; re-ingest is destroy-after-
confirm; `indexing.wasb` config
9       > real; eval gate hardened (tracked baseline, exit-3 on missing, GT-hash); API path
validation; WSL/provider timeouts +
10      > SQLite WAL; collector licensing gate + golden-label import guard + CVAT gate fixes;
**corpus reclassified** (23
11      > ShuttleSet broadcast rows demoted to non-commercial ? the commercial export is now
owned-data-only); docs truth-synced.
12      > ? **Metric corrections re-validated the quality story and strengthened it: learned
LOVO 0.583 ? 0.626 vs heuristic
13      > 0.480** (midrank AUC + fps-normalized speed; per-video AUC unchanged 0.83?0.92).
14
15      ## The recommendation (one paragraph)
16      Build the **quality-first extraction harness** before anything else: a learned **TAL
head** (ActionFormer/ASFormer, MIT)
17      over trajectory features is the robust path ? it **decouples from the multi-court
"background games" confound** that the
18      unsupervised heuristic (and audio, and spatial court-gating) can't escape (all three
tested + failed; it's a *temporal*
19      rally-vs-non-rally problem). **Framework = ms-swift, base = Qwen3-VL** (Gemma-4
AMBER/bench) come **later** for the
20      conversational/grounding upgrade ? start with the head. **On-device is deferred**
(export-clean ops now so it stays
21      reachable). Everything stays **MIT/Apache-2.0/BSD** (code + data + weights). Bottleneck
= **data + per-sport trajectory
22      detectors**, not model architecture.
```

23
24      ## Latest findings (2026-06-09)
25      - **6-match badminton LOVO:** learned prototype **0.583** vs heuristic **0.480**
(learned leads since n=3, AUC 0.83?0.92,
26        decoupled from multi-court). The "contrast" metric (in-rally÷dead-time visibility)
predicts heuristic F1 monotonically.
27      - **?? Multisport (Roora pickleball + TT):** labels **ingest cleanly** (`curate convert-
cvat` ? 6 rallies/sport, sane).
28        **WASB-transfer SURPRISE (directional, n=6 each ? TINY):** the badminton shuttle
detector segments **TABLE TENNIS at
29        F1 0.909** out-of-the-box (!) but **pickleball only 0.308**. ? **The per-sport-
detector blocker is SPORT-DEPENDENT:** TT may
30        be servable with WASB short-term; pickleball needs a proper (MIT/Apache) ball
detector. Confirm/deny with more videos.
31      - **Owner field notes (2026-06-09):** camera height/distance **varied on purpose**
(great generalization data); **multi-court
32        adjacent-court SOUND dominates the signal** ? empirically reconfirms audio-foreground
isolation won't work + the
33        robustness-first learned-model path. Multi-court is the realistic (academy)
environment, so this *is* the target distribution.
34
35      ## Prioritized next steps
36      1. **[owner greenlight] Phase-0 (M0.x), $0/local:**
37         - **M0.4 ? Gen-0 TAL harness** (first-class): productionize `rally_seq_proto.py` ?
one-command train?eval?**ship-gate** over
38           ActionFormer/ASFormer, reusing `metrics.match_intervals`. **Define the ship-gate
target** (explicit F1@tIoU; floor = beat
39           heuristic LOVO **0.480**; the corrected learned prototype already sits at
**0.626**) ? "very high P/R" must be
40           operationalized, not just "beat the floor." Add **paired per-video stats**
(sign/Wilcoxon) ? at n=6 a mean gap alone is noise-ambiguous.
41         - **M0.1 ? corpus spine + the per-video event-index contract** (the conversational
seam: extract ? event store ? text-to-SQL).
42           Includes importing the n=6 golden matches into the collector (now safe: import
guard + `--normalize`, PR #15).
43         - **M0.3 ? compliance:** ? corpus licensing reclass DONE (collector #14/#17 ?
ShuttleSet demoted, export owned-only).
44           REMAINING: retire/retarget the Gemini-silver training path
(`backend/eval/distill_local.py` + `calibrate_local`
45           thresholds; artifacts in gitignored `output/`); WASB **C3** own-weights/clean-swap
(gates ship; un-scoped new
46           workstream ? temporal labels can't train the detector, per ADR-002).
47      2. **Multisport thread:**
48         - Ingest the Roora pickleball/TT rally CSVs into the corpus (collector `import-local
--normalize`).
49         - **Get more pickleball + TT videos** ? re-run the WASB-transfer eval to confirm/deny
TT?0.91, pickleball?0.31 (n=6 is too few).
50         - Once each sport has ?3 matches, run a **merged prototype LOVO across badminton + TT
(+ pickleball)** ? the "how would a
51           merged model do" experiment. Identify a clean MIT/Apache ball detector for sports
where WASB doesn't transfer (pickleball).
52           ? Any TT-on-WASB serving is still gated by C3 (checkpoint provenance multiplies per
sport).
53      3. **PMF validation (cheaper than any engineering month ? audit finding: the "India
badminton whitespace" was falsified;
54         Bangalore has ?4 funded players; the surviving wedge = software-only multi-court BYO-
footage indexing):**
55         - **C13 discovery interviews** (10?15 Bengaluru academies) + two added questions: "do
you have multi-court footage
56           nobody watches?" and "would you pay per-match to have it indexed?"
57         - **Gemini-wrapper baseline vs the n=6 LOVO corpus** (harness exists) ? if a wrapper
? learned 0.626, the owned-model
58           investment is premature and the play is distribution/UX.
59      4. **Housekeeping:** **rename repo off "badminton"** (multisport, owner action ? now in
BACKLOG); confirm collector =

```
60         corpus/event-store system-of-record.
61
62      ## Pointers / artifacts
63      - Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Study:
`docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`
64        (2026-06-09 banner) · Quality: `docs/archives/quality-iterations/2026-06-multivideo-
lovo/`.
65      - Eval corpus: `output/human_lovo_manifest.json` (badminton n=6; gitignored ? back it up
with the artifacts). **Durable
66        artifacts: `C:\Users\avidu\Projects\Annotation Setup\Collect\Trajectories\`** (all
WASB trajectories + `roora\`
67        pickleball/TT CSVs+XMLs ? rescued from `%TEMP%`). Labels: `Annotation
Setup\Collect\Golden Labelled\`.
68        Code: `backend/eval/{eval_partial_labels,rally_seq_proto,calibrate_local}.py`.
69      - Collector corpus backup (pre-reclass): `sports-data-
collector/data/sports_metadata.backup-2026-06-10.db` (gitignored).
70      - ? No owned-model implementation until owner greenlights a milestone. Committed next
PLATFORM slice = async job model (single-tenant).
71
```

**12. user (2026-06-10T09:33:56.791Z)**

```
1       """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2       on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3       100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5       Pipeline being tuned (all local, no Gemini at runtime):
6           WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7                                -> rally_gate (min_crossings)            -> rally windows
8
9       Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10      videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11      the honest "how precise is the local pipeline on an unseen video" number. The
12      output is a single local config + the precision it buys, with NO Gemini dependency
13      shipped.
14
15      Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16      calibration.{select_best, leave_one_video_out, improvement_report}.
17
18      Run:
19          python -m backend.eval.calibrate_local --manifest videos.json
20      where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21      (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Callable, Dict, List, Tuple
28
29      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30      from backend.pipeline.detectors import rally_gate
31      from backend.eval.calibrate_wasb import load_golden_gts
32      from backend.eval.calibration import select_best, leave_one_video_out,
improvement_report, evaluate
33
34      # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
35      LocalConfig = Tuple[float, float, float, int]
36      BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0)   # today's local default (windowing,
no gate)
37
38
39      def make_local_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[LocalConfig], list]:
40          """predict_fn(cfg) -> rally windows from a cached WASB trajectory, LOCAL ONLY
```

```
        (windowing thresholds + the shuttle net-crossing gate). Parsed once."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        def predict_fn(cfg: LocalConfig) -> List[Tuple[float, float]]:
            vt, mg, mwd, mc = cfg
            wins = TrackNetRunner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width,
                velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            )
            wins = [(w["start"], w["end"]) for w in wins]
            if mc > 0:
                wins = rally_gate.apply_gate(points, wins, fps, min_crossings=mc)
            return wins

        return predict_fn


    def local_grid() -> List[LocalConfig]:
        return [(vt, mg, mwd, mc)
                for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
```

**13. user (2026-06-10T09:33:57.219Z)**

```
    """Evaluate the rally pipeline against PARTIAL human golden labels.

    The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
    ending_reason,sport). While a video is only partially tagged, the labels cover a
    PREFIX of the video ? every rally up to some point is marked, and everything after is
    simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
    the detector's *correct* rallies in the un-labeled tail would be counted as false
    positives and crush precision.

    This harness fixes that by restricting the evaluation to the LABELED WINDOW
    [window_start, window_end] (window_end defaults to the last labeled rally's end) and
    dropping predicted segments that fall outside it. Within that window the human labels
    ARE complete, so precision/recall/F1 are honest.

    It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
      - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
      - pipeline.detectors.rally_gate.apply_gate   (net-crossing "gate A")
      - eval.metrics.match_intervals / score
      - export_wasb_segments.build_comparison / write_comparison_csv  (per-rally detail CSV)

    Run:
        python -m backend.eval.eval_partial_labels \
            --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
            --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
Avi.rallies.csv" \
            --fps 59.94 --frame-width 1920
    """
    from __future__ import annotations

    import argparse
    import os.path as osp
    from typing import List, Optional, Tuple

    from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
    from backend.eval.metrics import score
    from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.pipeline.detectors import rally_gate

    Interval = Tuple[float, float]
    Config = Tuple[float, float, float]

```

```
42      # Operating points to compare. The tuned config wins on its fit-video but over-segments
43      # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44      TUNED: Config = (0.05, 1.0, 1.0)
45
46
47      def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
48          """Keep intervals that overlap the labeled window [w0, w1].
49
50          A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51          in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52          they are neither rewarded nor penalized. GTs are assumed already inside the window.
53          """
54          return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55
56
57      def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
58                      window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59          """Score one (config, gate) operating point inside the labeled window."""
60          preds = predict_fn(cfg)
```

**14. user (2026-06-10T09:34:07.697Z)**

```
Found 9 files
docs\NEXT_STEPS.md
docs\CODE_MAP.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
tests\test_rally_seq_proto.py
backend\eval\rally_seq_proto.py
scratch\HIGH_RISK_REMEDIATION_PLAN.md
docs\archives\quality-iterations\README.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
```

**15. user (2026-06-10T09:34:08.942Z)**

```
236:                        start_time=_safe_float(obj.get("start_time")),
237:                        end_time=_safe_float(obj.get("end_time")),
248:                        start_time=_safe_float(obj.get("start_time")),
249:                        end_time=_safe_float(obj.get("end_time")),
260:                        start_time=_safe_float(obj.get("start_time")),
261:                        end_time=_safe_float(obj.get("end_time")),
272:                        start_time=_safe_float(obj.get("start_time")),
273:                        end_time=_safe_float(obj.get("end_time")),
284:                        start_time=_safe_float(obj.get("start_time")),
285:                        end_time=_safe_float(obj.get("end_time")),
303:                            start_time=_safe_float(row.get("start_time")),
304:                            end_time=_safe_float(row.get("end_time")),
315:                            start_time=_safe_float(row.get("start_time")),
316:                            end_time=_safe_float(row.get("end_time")),
327:                            start_time=_safe_float(row.get("start_time")),
328:                            end_time=_safe_float(row.get("end_time")),
339:                            start_time=_safe_float(row.get("start_time")),
340:                            end_time=_safe_float(row.get("end_time")),
351:                            start_time=_safe_float(row.get("start_time")),
352:                            end_time=_safe_float(row.get("end_time")),
552:        indexer's `start,end` ground-truth format (decimal seconds), plus a single
844:                    sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
850:                    sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
856:                    sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
886:        start_time = None
887:        end_time = None
```

```
899:                     start_time = rally.start_time
900:                     end_time = rally.end_time
913:                     start_time = pt.start_time
914:                     end_time = pt.end_time
929:                     start_time = d.start_time
930:                     end_time = d.end_time
935:         if start_time is None or end_time is None:
939:         print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
945:             # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
946:             play_url = f"file:///{abs_path.replace(os.sep, '/')}#t={start_time},{end_time}"
950:                 t_sec = int(start_time)
```

**16. user (2026-06-10T09:34:09.783Z)**

```
80
81          def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82              return (self.predict_proba(X) >= threshold).astype(int)
83
84          # --- persistence (small JSON, no pickle/joblib) ---------------------------
85
86          def to_dict(self) -> dict:
87              return {
88                  "model": "logreg",
89                  "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90                  "w": self.w.tolist() if self.w is not None else None,
91                  "b": self.b,
92                  "mean": self.mean.tolist() if self.mean is not None else None,
93                  "std": self.std.tolist() if self.std is not None else None,
94                  "feature_names": self.feature_names,
95              }
96
97          @classmethod
98          def from_dict(cls, d: dict) -> "LogisticRegression":
99              m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100             m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101             m.b = float(d.get("b", 0.0))
102             m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103             m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104             m.feature_names = d.get("feature_names")
105             return m
106
107         def save(self, path: str) -> None:
108             with open(path, "w") as f:
109                 json.dump(self.to_dict(), f, indent=2)
110
111         @classmethod
112         def load(cls, path: str) -> "LogisticRegression":
113             with open(path) as f:
114                 return cls.from_dict(json.load(f))
115
```

**17. user (2026-06-10T09:34:17.137Z)**

```
1       """Smoke + sanity for the learned per-frame rally segmenter prototype.
2
3       Not a quality bar (it's a prototype) ? just pins that the feature/label/segment plumbing
4       works and that a clearly rally-vs-dead synthetic trajectory is separable (AUC well above
5       chance), so the proof-of-signal claim is reproducible.
6       """
7       import csv
8
9       import numpy as np
10      import pytest
```

```
11
12      from backend.eval import rally_seq_proto as rp
13
14
15      def _write_traj(path, fps=10.0):
16          """120s synthetic trajectory: shuttle oscillates fast across the net during
'rallies'
17          (0-20s, 40-60s, 80-100s) and is mostly still/invisible in the dead time between."""
18          rows = [("Frame", "Visibility", "X", "Y")]
19          n = int(120 * fps)
20          for f in range(n):
21              t = f / fps
22              in_rally = (t < 20) or (40 <= t < 60) or (80 <= t < 100)
23              if in_rally:
24                  x = 50 + 45 * np.sin(2 * np.pi * (f % 10) / 10.0)  # crosses net (x=50)
repeatedly
25                  rows.append((f, 1, round(float(x), 2), 50))
26              else:
27                  if f % 5 == 0:                                      # mostly invisible,
occasional still blip
28                      rows.append((f, 1, 50.0, 50))
29                  else:
30                      rows.append((f, 0, -1, -1))
31          with open(path, "w", newline="") as fh:
32              csv.writer(fh).writerows(rows)
33          return str(path)
34
35
36      def _write_labels(path):
37          with open(path, "w", newline="", encoding="utf-8") as fh:
38              w = csv.writer(fh)
39              w.writerow(["rally_number", "start_time", "end_time"])
40              w.writerow([1, 0.0, 20.0]); w.writerow([2, 40.0, 60.0]); w.writerow([3, 80.0,
100.0])
41          return str(path)
42
43
44      def test_features_and_labels_align(tmp_path):
45          from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
46          pts = TrackNetRunner.parse_trajectory_csv(_write_traj(tmp_path / "t.csv", fps=10.0))
47          arr = rp.build_frame_arrays(pts, frame_width=100.0, fps=10.0)
48          X, times = rp.features(arr, fps=10.0)
49          assert X.shape[1] == len(rp.FEAT_NAMES) and X.shape[0] == len(times)
50          assert np.isfinite(X).all()
51          y = rp.labels_for(times, [(0.0, 20.0), (40.0, 60.0), (80.0, 100.0)])
52          assert 0 < y.sum() < len(y)            # both classes present
53
54
55      def test_signal_is_separable(tmp_path):
56          """The whole point: rally vs dead-time is separable from the trajectory features."""
57          res = rp.run([{
58              "name": f"v{i}", "trajectory": _write_traj(tmp_path / f"t{i}.csv", fps=10.0),
59              "fps": 10.0, "frame_width": 100.0, "labels": _write_labels(tmp_path /
f"g{i}.csv"),
60          } for i in range(2)])
```

**18. user (2026-06-10T09:34:18.087Z)**

```
15
16      ## Why this repo matters
17
18      The indexer's best rally mode refines local detections with a **paid Gemini API**.
19      The strategic goal is to replace that with an **offline model trained on our own
20      commercially-licensed, annotated corpus** ? making rally detection free, offline,
21      and self-improving. None of that is possible without a clean, licensable,
```

```
accurately-labelled, full-resolution dataset. That dataset is what this repo
produces.

## What the two key commands do (and why)

- **`curate export`** ? emits the indexer's ground-truth format: per-video
  `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
  Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
  table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
  `is_commercial`, so only licensable data flows
  downstream. The annotations are **temporal-only** ? they train/evaluate rally
  *segmentation*, not the frame-level shuttle detector (see ADR-002).

- **`fetch`** ? (re)downloads videos at **best available resolution**, updating
  the DB **in place** so annotations and curation status are never lost. Uses
  authenticated cookies for reliable, unthrottled downloads. The why and the
  hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.

## The one contract that matters: video identity

The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
exact `local_path`, and any downstream processing must run on **that** file. This is
```

**19. user (2026-06-10T09:34:18.570Z)**

```
"""Batch evaluation across a collector export manifest.

Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
sports-data-collector's `curate export`, (optionally) process each video through
a segmenter so detections land in the indexer DB, then score every video's
detected rallies against its ground-truth CSV and aggregate the results.

This module holds the *pure* pieces (path resolution, stage selection, metric
aggregation) so they can be unit-tested without touching video files, the GPU,
or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
"""

import os
from typing import Dict, List, Optional


# Segmenters that only ever populate the `final` play_segments table (no raw
# candidate stage), so scoring `candidates` for them would be meaninglessly empty.
_FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}


def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
    """Resolve a manifest `local_path` to an absolute path.

    The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
    to its own repo root and with Windows separators. Normalise separators, strip
    a leading ``./``, and resolve relative paths against `videos_root` (the
    collector root). Returns None for a missing/empty path.
    """
    if not local_path:
        return None
    p = local_path.replace("\\", "/")
    if p.startswith("./"):
        p = p[2:]
    # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
    # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
    # This keeps collector manifest paths working in tests and mixed environments.
    if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
        return os.path.normpath(p)
```

```
        return os.path.normpath(os.path.join(videos_root, p))


    def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
        """Pick which prediction stages to score.

        ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
        but only ``final`` for segmenters that don't emit candidates. An explicit
        request ('candidates' | 'final' | 'both') overrides the heuristic.
        """
        if requested == "both":
            return ["candidates", "final"]
        if requested in ("candidates", "final"):
            return [requested]
        # auto
        if segmenter in _FINAL_ONLY_SEGMENTERS:
            return ["final"]
        return ["candidates", "final"]


    def _safe_div(num: float, den: float) -> float:
```

**20. user (2026-06-10T09:34:30.087Z)**

```
    """Per-video observability reports for the indexer.

    `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
    no-op and the pipeline runs exactly as before. The single entrypoint
    `emit_indexer_report(...)` is designed to be called from a `finally` block and
    NEVER raises ? emitting a report must not be able to break a processing run.
    """
    from __future__ import annotations

    import os
    from typing import Any, Dict, List, Optional

    try:  # soft dependency ? never hard-fail the pipeline on a reporting import
        import obsreport  # noqa: F401
        from . import harvest

        OBSREPORT_AVAILABLE = True
    except Exception:  # pragma: no cover - exercised only when obsreport is absent
        OBSREPORT_AVAILABLE = False


    def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
        return bool((config or {}).get("report", {}).get("enabled", True))


    def emit_indexer_report(
        *,
        db,
        video_id: str,
        video_path: Optional[str],
        config: Optional[Dict[str, Any]],
        val_results: List[Any],
        val_context: Optional[Dict[str, Any]] = None,
        segmenter_requested: Optional[str] = None,
        segmenter_actual: Optional[str] = None,
        was_reingest: bool = False,
        segmentation_ran: bool = True,
    ) -> Optional[Dict[str, str]]:
        """Harvest + write the report. Returns the written file paths, or None if
        reporting is unavailable/disabled/failed. Never raises."""
        config = config or {}
```

```
42          # Accept either a plain dict or a typed config object (e.g.
backend.config.AppConfig).
43          # Normalize to a plain nested dict so harvest's dict access works uniformly.
44          if hasattr(config, "model_dump"):
45              try:
46                  config = config.model_dump(mode="json")
47              except Exception:
48                  config = {}
49          if not OBSREPORT_AVAILABLE or not report_enabled(config):
50              return None
51          try:
52              play_segments = db.query_play_segments(video_id, {}) if db is not None else []
53              candidates = db.query_candidate_segments(video_id) if db is not None else []
54              rec = harvest.record_run(
55                  video_id=video_id,
56                  video_path=video_path,
57                  config=config,
58                  val_results=val_results,
59                  val_context=val_context,
60                  play_segments=play_segments,
61                  candidates=candidates,
62                  segmenter_requested=segmenter_requested,
63                  segmenter_actual=segmenter_actual,
64                  was_reingest=was_reingest,
65                  segmentation_ran=segmentation_ran,
66              )
67              output_dir = (config.get("report") or {}).get("output_dir")
68              res = rec.write(output_dir=output_dir)
69              return res.paths or None
70          except Exception:  # pragma: no cover - defensive; reporting must never break a run
71              return None
72
```

**21. user (2026-06-10T09:34:31.029Z)**

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
```

```
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38          sport: str = "badminton"
39          wsl_stage_dir: str = "~/clips"
40          timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41
42          @classmethod
43          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
44              w = (idx_cfg or {}).get("wasb", {}) or {}
45              return cls(
46                  distro=w.get("wsl_distro", "Ubuntu"),
47                  repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
48                  conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
49                  conda_env=w.get("conda_env", "wasb"),
50                  weights_path=w.get("weights_path",
```

**22. user (2026-06-10T09:34:31.720Z)**

```
tests\test_import_smoke.py
```

**23. user (2026-06-10T09:34:50.205Z)**

```
33:    if p.startswith("./"):
```

**24. user (2026-06-10T09:34:51.265Z)**

```
<tool_use_error>Path does not exist: C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\eval\run_phase3_eval.py. Note: your current working directory is
C:\Users\avidu\Projects\badminton-highlight-indexer.</tool_use_error>
```

**25. user (2026-06-10T09:34:56.608Z)**

```
backend\eval\__init__.py
backend\eval\annotations.py
backend\eval\classifier.py
backend\eval\training.py
backend\eval\calibration.py
backend\eval\gemini_refine.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\distill_local.py
backend\eval\export_wasb_segments.py
backend\eval\harness.py
backend\eval\batch.py
backend\eval\eval_partial_labels.py
backend\eval\metrics.py
backend\eval\rally_seq_proto.py
backend\eval\regression.py
```

**26. user (2026-06-10T09:35:04.173Z)**

```
backend\eval\batch.py:1:"""Batch evaluation across a collector export manifest.
backend\eval\batch.py:3:Phase 3 of the collector->indexer bridge: walk the `manifest.json`
emitted by
backend\eval\batch.py:23:    """Resolve a manifest `local_path` to an absolute path.
backend\eval\batch.py:37:    # This keeps collector manifest paths working in tests and mixed
```

```
environments.
backend\eval\calibrate_wasb.py:34:def load_golden_gts(csv_path: str) -> List[Interval]:
backend\eval\calibrate_wasb.py:76:    gts = load_golden_gts(labels_csv)
backend\eval\calibrate_local.py:15:Reuses: TrackNetRunner windowing, rally_gate,
calibrate_wasb.load_golden_gts,
backend\eval\calibrate_local.py:19:    python -m backend.eval.calibrate_local --manifest
videos.json
backend\eval\calibrate_local.py:31:from backend.eval.calibrate_wasb import load_golden_gts
backend\eval\calibrate_local.py:70:        gts = load_golden_gts(s["labels"])
backend\eval\calibrate_local.py:115:    ap.add_argument("--manifest", required=True,
backend\eval\calibrate_local.py:120:    with open(args.manifest, encoding="utf-8") as f:
backend\eval\export_wasb_segments.py:27:from backend.eval.calibrate_wasb import make_predict_fn,
load_golden_gts, BASELINE
backend\eval\export_wasb_segments.py:172:        gts = load_golden_gts(labels_csv)
backend\eval\distill_local.py:18:rally_gate, metrics, calibrate_wasb.load_golden_gts,
gemini_refine.merge_overlaps.
backend\eval\distill_local.py:21:    python -m backend.eval.distill_local --manifest videos.json
--model-out output/local_rally_model.json
backend\eval\distill_local.py:33:from backend.eval.calibrate_wasb import load_golden_gts
backend\eval\distill_local.py:84:    gts = load_golden_gts(spec["labels"])
backend\eval\distill_local.py:146:    ap.add_argument("--manifest", required=True,
backend\eval\distill_local.py:152:    with open(args.manifest, encoding="utf-8") as f:
backend\eval\eval_partial_labels.py:16:  - calibrate_wasb.make_predict_fn / load_golden_gts /
BASELINE
backend\eval\eval_partial_labels.py:33:from backend.eval.calibrate_wasb import make_predict_fn,
load_golden_gts, BASELINE
backend\eval\eval_partial_labels.py:72:    gts_all = load_golden_gts(labels_csv)
backend\eval\eval_partial_labels.py:153:    gts_all = load_golden_gts(labels_csv)
backend\eval\audio\e1_probe.py:9:Reuses: audio.hit_detector,
calibrate_wasb.load_golden_gts/make_predict_fn,
backend\eval\audio\e1_probe.py:19:from backend.eval.calibrate_wasb import load_golden_gts,
make_predict_fn, BASELINE
backend\eval\audio\e1_probe.py:56:    gts = load_golden_gts(labels_csv)
backend\eval\rally_seq_proto.py:26:    python -m backend.eval.rally_seq_proto --manifest
output/human_lovo_manifest.json
backend\eval\rally_seq_proto.py:40:from backend.eval.calibrate_wasb import load_golden_gts
backend\eval\rally_seq_proto.py:176:        gts = load_golden_gts(s["labels"])
backend\eval\rally_seq_proto.py:214:    ap.add_argument("--manifest", required=True, help="JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
backend\eval\rally_seq_proto.py:216:    with open(args.manifest, encoding="utf-8") as f:
backend\pipeline\detectors\wasb_infer.py:23:    records progress in ``<cache>/manifest.json``
(atomic write). A reboot loses at
backend\pipeline\detectors\wasb_infer.py:52:_MANIFEST_FILE = "manifest.json"
backend\pipeline\detectors\wasb_infer.py:114:def write_manifest(cache_dir: str, manifest: dict)
-> None:
backend\pipeline\detectors\wasb_infer.py:116:    _atomic_write_text(mpath, json.dumps(manifest,
indent=2))
backend\pipeline\detectors\wasb_infer.py:119:def read_manifest(cache_dir: str):
backend\pipeline\detectors\wasb_infer.py:130:def manifest_compatible(manifest: dict, *,
frames_dir: str, weights: str, sport: str,
backend\pipeline\detectors\wasb_infer.py:133:    if not manifest or manifest.get("version") !=
CACHE_VERSION:
backend\pipeline\detectors\wasb_infer.py:136:        manifest.get("frames_dir") ==
osp.abspath(frames_dir)

[Showing results with pagination = limit: 40]
```

**27. user (2026-06-10T09:35:18.619Z)**

```
run_phase3_eval.py
```

**28. user (2026-06-10T09:35:19.283Z)**

```
No files found
```

**29. user (2026-06-10T09:35:25.188Z)**

```
44:def build_parser() -> argparse.ArgumentParser:
75:def main(argv=None) -> int:
```

**30. user (2026-06-10T09:35:30.317Z)**

```
1       """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3       Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4       processes each video through a segmenter (so detections land in the indexer DB),
5       scores each against its ground-truth rally CSV, and prints per-video plus
6       corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8       Examples
9       --------
10      Free CPU baseline on a single video (validate the loop):
11          python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
12              --segmenter motion --only shuttleset_1
13
14      Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
15          python run_phase3_eval.py --manifest .../manifest.json --segmenter yolo_hybrid \
16              --json output/phase3_report.json
17
18      Re-score only (videos already processed), no re-segmentation:
19          python run_phase3_eval.py --manifest .../manifest.json --score-only
20      """
21
22      import os
23      import sys
24      import json
25      import time
26      import argparse
27      import logging
28
29      from dotenv import load_dotenv
30      load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
31
32      from backend.infrastructure.database import Database
33      from backend.config import load_config
34      from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
35      # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36      from backend.pipeline.segmenters import segmenter_registry
37      from backend.eval.annotations import load_annotations
38      from backend.eval.harness import evaluate, format_report_text, report_to_dict
39      from backend.eval import batch
40
41      logger = logging.getLogger(__name__)
42
43
44      def build_parser() -> argparse.ArgumentParser:
45          p = argparse.ArgumentParser(
46              description="Batch process + evaluate a collector export manifest (Phase 3).",
47              formatter_class=argparse.RawDescriptionHelpFormatter,
48          )
49          p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
50          p.add_argument("--videos-root", default=None,
51                         help="Root for resolving manifest relative local_paths "
52                              "(default: the collector repo root, two levels above the
manifest).")
53          p.add_argument("--segmenter", default="motion",
54                         help="Segmenter to run (default: motion ? free/CPU). "
55                              "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
```

```
56          p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
default="auto",
57                         help="Which prediction stage(s) to score (default: auto by
segmenter).")
58          p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
59          p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
60          p.add_argument("--only", action="append", default=None,
61                         help="Only this video_id (repeatable).")
62          p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
63          p.add_argument("--max-frames", type=int, default=None,
64                         help="Cap segmenter frames (fast validation).")
65          p.add_argument("--score-only", action="store_true",
66                         help="Skip segmentation; only score detections already in the DB.")
67          p.add_argument("--reprocess", action="store_true",
68                         help="Re-run segmentation even if the video already has segments.")
69          p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
70          p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
71          p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
72          return p
73
74
75      def main(argv=None) -> int:
```

**31. user (2026-06-10T09:35:37.860Z)**

```
1       """Load hand-annotated ground-truth rally timestamps from CSV.
2
3       Annotation file format (one file per video):
4
5           start,end
6           00:01:12,00:01:39
7           102.5,131.0
8
9       - A header row `start,end` is optional (auto-detected and skipped).
10      - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11        `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12      - Blank lines and `#` comment lines are ignored.
13      """
14
15      import csv
16      import logging
17      from typing import List, Tuple
18
19      logger = logging.getLogger(__name__)
20
21      Interval = Tuple[float, float]
22
23
24      def parse_timestamp(value: str) -> float:
25          """Parse a timestamp string to float seconds.
26
27          Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28          Raises ValueError on anything unparseable so bad annotations fail loudly.
29          """
30          value = value.strip()
31          if not value:
32              raise ValueError("empty timestamp")
33          if ":" in value:
34              parts = value.split(":")
35              if len(parts) > 3:
```

```
36                  raise ValueError(f"too many ':' segments in timestamp '{value}'")
37              total = 0.0
38              for part in parts:                # most-significant first (H, M, S)
39                  total = total * 60 + float(part)
40              return total
41          return float(value)
42
43
44      def load_annotations(csv_path: str) -> List[Interval]:
45          """Load and validate ground-truth rally intervals from a CSV file.
46
47          Returns intervals sorted by start time. Raises ValueError on malformed rows
48          (bad timestamp, start >= end) so labeling mistakes surface immediately rather
49          than silently skewing scores.
50          """
51          intervals: List[Interval] = []
52          with open(csv_path, newline="") as f:
53              reader = csv.reader(f)
54              for lineno, row in enumerate(reader, start=1):
55                  if not row:
56                      continue
57                  first = row[0].strip()
58                  if not first or first.startswith("#"):
59                      continue
60                  # Skip an optional header row.
61                  if first.lower() in ("start", "start_time"):
62                      continue
63                  if len(row) < 2:
64                      raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
65                  try:
66                      start = parse_timestamp(row[0])
67                      end = parse_timestamp(row[1])
68                  except ValueError as e:
69                      raise ValueError(f"{csv_path}:{lineno}: {e}") from e
70                  if start >= end:
71                      raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end})")
72                  intervals.append((start, end))
73
74          intervals.sort(key=lambda iv: iv[0])
75          logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
76          return intervals
77
78
79      def write_scaffold(csv_path: str) -> None:
80          """Write an empty annotation file with just the header, for hand-filling."""
81          with open(csv_path, "w", newline="") as f:
82              writer = csv.writer(f)
83              writer.writerow(["start", "end"])
84
```

**32. assistant (2026-06-10T09:37:27.988Z)**

All load-bearing citations verified across both working trees. Cross-examination complete ? delivering the structured findings.

**33. user (2026-06-10T09:38:39.444Z)**

Structured output provided successfully

**34. assistant (2026-06-10T09:38:51.245Z)**

Cross-examination delivered via structured output. Key verified findings: Paper A's obsreport precedent is a commented-out pin that has never been live (C:\Users\avidu\Projects\badminton-

highlight-indexer\requirements.txt lines 12-19); the golden-CSV header fork is still live and exists *within* the indexer itself (backend\eval\annotations.py vs backend\eval\calibrate_wasb.py), cutting against both A's "contracts already worked" and B's "co-location prevents drift"; Paper B's "ship-gate needs the DB" conflates the regression gate with the file-shaped LOVO model-selection gate in backend\eval\rally_seq_proto.py; and Paper C's eval_baselines claim checks out (nothing committed). Verdict: C-shaped hybrid (in-repo `training/` now, executable import boundary, slim artifact manifest, named Gen-2 split destination), with the golden-label/corpus contract as the single thing to freeze first.